

SVEUČILIŠTE U ZAGREBU

PRIRODOSLOVNO-MATEMATIČKI FAKULTET

MATEMATIČKI ODSJEK

Marko Cindrić

**Numeričko rješavanje problema optimalnog transporta
metodom proksimalnog cijepanja na grafičkoj kartici**

Zagreb, 2026.

Ovaj rad izrađen je na Matematičkom odsjeku Prirodoslovno-matematičkog fakulteta pod vodstvom izv. prof. dr. sc. Marija Bukala te je predan na natječaj za dodjelu Rektorove nagrade u akademskoj godini 2025./2026.

Sadržaj

1. Uvod	1
2. Optimalni transport	4
2.1. Mongeova formulacija	4
2.2. Kantorovicheva formulacija	5
2.3. Dinamička formulacija	6
3. Diskretizacija	9
3.1. Centrirana mreža	9
3.2. Pomaknuta mreža	9
3.3. Pomoćne definicije	10
3.4. Diskretizacija kontinuiranog problema	12
4. Douglas-Rachfordov algoritam	15
4.1. Proksimalni operator od $\mathcal{J}_d$	17
4.2. Proksimalni operator od ι_{C_d}	18
4.3. Proksimalni operator od $\iota_{C_{c,s}}$	20
4.4. Definicija iteracija	22
5. Implementacija	25
5.1. Centrirana i pomaknuta mreža	25
5.2. Implementacija proksimalnog operatora od $\mathcal{J}_d$	26
5.2.1. Rješavanje kubne jednadžbe	26
5.2.2. Newtonova metoda	27
5.3. Implementacija proksimalnog operatora od ι_{C_d}	28
5.3.1. Izvod izraza za AA^*	28

5.3.2.	Kod za osnovne operatore	32
5.3.3.	Spektralna dekompozicija operatora L	35
5.3.4.	Rješavanje sustava $\operatorname{div} _{\operatorname{Im} M}(\operatorname{div}^* _{\operatorname{Im} M}(\lambda)) = \operatorname{div}(\tilde{U}_{cl})$	36
5.3.5.	Diskretna kosinusova transformacija	39
5.3.6.	Opis implementacije proksimalnog operatora	43
5.4.	Implementacija proksimalnog operatora od $\iota_{C_{c,s}}$	44
5.4.1.	Implementacija operatora $\mathcal{I}$	44
5.4.2.	Implementacija operatora $\mathcal{I}^*$	46
5.4.3.	Izvod izraza za BB^*	48
5.4.4.	Thomasov algoritam	50
5.4.5.	Opis implementacije proksimalnog operatora	52
5.5.	Implementacija iteracije Douglas-Rachfordovog algoritma	52
5.6.	Implementacija petlje Douglas-Rachfordovog algoritma	55
6.	Numeričke simulacije	60
6.1.	Specifikacija računala	60
6.2.	Validacija algoritma	60
6.3.	Usporedba s postojećim implementacijama	63
6.4.	Različiti primjeri transporta	64
6.4.1.	Primjer cijepanja gustoće	64
6.4.2.	Rukom crtane gustoće	65
6.4.3.	Neravnomjerno raspoređene gustoće	65
6.4.4.	Transport između dviju crno-bijelih slika	65
7.	Zaključak	68
	Literatura	70
	Sažetak	72
	Abstract	73

1. Uvod

Zamislamo hrpu pijeska i jamu koju tom hrpom treba zatrpati. Pijeska ima točno onoliko koliko jama može primiti, ali svaka lopata pijeska mora negdje otputovati, a taj put nešto košta. Pitanje je jednostavno za postaviti i iznenađujuće teško za odgovoriti: kako rasporediti pijesak tako da ukupni trošak prijevoza bude najmanji? Nije riječ o tome koliko pijeska treba prenijeti, jer taj je iznos zadan, nego o tome što ide kamo.

Upravo je to pitanje 1781. godine članovima Pariške akademije postavio francuski matematičar Gaspard Monge, razmišljajući o iskopima i nasipima pri gradnji utvrda [1]. Mongeova je formulacija problema prirodna, ali i kruta: ona svakoj čestici pijeska pridružuje točno jedno odredište. Time se, međutim, unaprijed isključuje mogućnost da se masa iz jedne točke razdijeli na više odredišta, a upravo je takvo cijepanje ponekad jedini način da se problem uopće riješi. Zbog toga Mongeov problem u općenitom slučaju ne mora imati rješenje, a njegova analiza dugo je ostala izvan dosega dostupnih tehnika.

Prekretnica je stigla tek stoljeće i pol kasnije, kada je Leonid Kantorovich četrdesetih godina prošlog stoljeća, motiviran problemima raspodjele resursa u planskoj ekonomiji, predložio bitno fleksibilniji pogled. Umjesto preslikavanja koje svaku točku šalje u jednu drugu točku, Kantorovich promatra transportne planove, mjere na produktu polazišnog i odredišnog prostora, koje opisuju koliko se mase seli s jednog mjesta na drugo. Time problem postaje, uz blage pretpostavke, uvijek rješiv. Za doprinose teoriji optimalne raspodjele resursa Kantorovich je 1975. dobio Nobelovu nagradu za ekonomiju.

Ono što je optimalni transport pretvorilo iz elegantne teorije u svakodnevni alat jest spoznaja da minimalni trošak transporta nije samo broj, nego udaljenost. Uz kvadratnu funkciju troška dobivamo Wassersteinovu udaljenost, metriku na prostoru vjerojatnosnih mjera koja, za razliku od klasičnih mjera razlike poput L^1 udaljenosti ili Kullback-Leiblerove divergen-

cije, uzima u obzir geometriju prostora na kojem mjere žive. Dvije uske, gotovo jednake gustoće koje se ne preklapaju za Wassersteinovu su udaljenost bliske ako su im nosači blizu, a za većinu drugih mjera jednako su daleke bez obzira na to jesu li razmaknute za milimetar ili kilometar. Ta je razlika presudna u primjenama. Sustavan pregled teorije daju Santambrogio [2] te Peyré i Cuturi [3].

Sredinom devedesetih Brenier je pokazao da je u kvadratnom slučaju optimalno preslikavanje gradijent konveksne funkcije, čime je optimalni transport povezan s Monge-Ampèreovom jednačbom i teorijom parcijalnih diferencijalnih jednačbi [4]. Nedugo zatim Benamou i Brenier [5] dali su formulaciju koja je temelj ovoga rada: umjesto da se pitamo tko ide kamo, pitamo se kako se masa kreće kroz vrijeme. Transport se opisuje gustoćom i poljem brzina koji zajedno zadovoljavaju jednačbu očuvanja mase, a minimizira se ukupna kinetička energija. Time se statički problem pretvara u problem dinamike fluida, a njegovo rješenje ne daje samo konačni raspored mase nego i cijelu putanju kojom je masa do njega došla. Zamjenom brzine količinom gibanja problem postaje konveksan, što ga čini dostupnim čitavom arsenalu metoda konveksne optimizacije.

Danas se optimalni transport pojavljuje na neočekivano mnogo mjesta. U računalnoj grafici i obradi slike koristi se za interpolaciju oblika, prijenos palete boja i usporedbu tekstura [6]. U medicinskoj slikovnoj dijagnostici služi za poravnavanje snimaka istog pacijenta u različitim trenucima te za usrednjavanje neuroloških snimaka [7]. U strojnom učenju Wassersteinova je udaljenost postala standardna funkcija gubitka pri učenju generativnih modela [8] i u obradi prirodnog jezika [9]. Vraća se i svojim ekonomskim korijenima, primjerice u teoriji aukcija [10].

Ta raširenost dolazi s cijenom. Numeričko rješavanje optimalnog transporta i dalje je računski zahtjevno, osobito u dinamičkoj formulaciji, u kojoj se dvodimenzionalnom prostornom problemu dodaje i vremenska dimenzija, pa broj nepoznanica raste kubno u finoći diskretizacije. Papadakis, Peyré i Oudet [11] pokazali su da se problem uspješno rješava metodama proksimalnog cijepanja, koje ciljni funkcional rastavljaju na nekoliko jednostavnih dijelova i svaki obrađuju zasebno [12]. Rezultat je algoritam čije se iteracije sastoje od velikog broja jednostavnih, međusobno neovisnih računa u svakoj točki mreže, uz nekoliko globalnih koraka koji se svode na klasične numeričke postupke. Drugim riječima, algoritam je gotovo idealan kandidat za izvođenje na grafičkoj kartici, no postojeće su implementacije,

uključujući onu autora ovog rada [13], pisane isključivo za procesor.

Cilj je ovoga rada iskoristiti tu priliku: implementirati algoritam proksimalnog cijepanja za dinamički optimalni transport u jeziku CUDA C++, tako da se cijeli račun izvodi na grafičkoj kartici, te izmjeriti koliko se time dobiva u odnosu na postojeća rješenja.

Rad je organiziran na sljedeći način. U drugom se poglavlju izlaže teorijska podloga, redom Mongeova, Kantorovicheva i dinamička formulacija problema, te se uvodi Wassersteinova udaljenost i konveksni oblik problema na kojem počiva ostatak rada. Treće poglavlje posvećeno je diskretizaciji: uvode se centrirana i pomaknuta mreža, pripadni diskretni operatori i diskretna inačica optimizacijskog problema. U četvrtom se poglavlju opisuje Douglas-Rachfordov algoritam i izvode tri proksimalna operatora koji odgovaraju trima dijelovima ciljnog funkcionala, nakon čega se zapisuju iteracije u obliku pogodnom za implementaciju. Peto je poglavlje najopsežnije i donosi detaljan opis implementacije, od organizacije podataka u memoriji i paraleliziranih kernela za pojedine operatore do korištenja diskretne kosinusove transformacije i CUDA grafova. U šestom se poglavlju provjerava ispravnost implementacije na primjerima za koje postoji analitičko rješenje, uspoređuje se brzina izvođenja s postojećim implementacijama te se prikazuje niz složenijih primjera transporta. Sedmo poglavlje sažima rezultate.

2. Optimalni transport

2.1. Mongeova formulacija

Označimo sa $\mathcal{P}(X)$ skup svih vjerojatnosnih mjera na X sa pripadnim σ -algebama. Dalje, neka su $\mu \in \mathcal{P}(X)$ izvorišna i $\nu \in \mathcal{P}(Y)$ odredišna vjerojatnosna mjera, a $c(x, y) = |x - y| : X \times Y \rightarrow [0, \infty)$ funkcija troška. Zatim promatramo familiju transportnih funkcija $T : X \rightarrow Y$, koje opisuju transport mase iz μ u ν . Točnije, za $\forall x \in X, T(x) \in Y$ određuje da se masa u točki x transportira u točku $y = T(x)$. Mongeov problem je optimizacijski problem u kojem je cilj naći transportni plan T koji minimizira ukupni trošak transporta μ na ν , odnosno, rješavamo problem

$$\inf_T \left\{ \int |x - T(x)| d\mu(x) : \mu(T^{-1}(A)) = \nu(A) \text{ za svaki izmjeriv skup } A \subset Y \right\}. \quad (2.1)$$

Napominjemo da je pri transportu masa očuvana i to je glavni razlog zašto bez smanjenja općenitosti možemo raditi s vjerojatnosnim mjerama.

U slučaju da su mjere apsolutno neprekinute u odnosu na Lebesgueovu mjeru na X i Y , odnosno da postoje funkcije gustoća f, g , uz uvjet da je T difeomorfizam, tj. bijektivna funkcija takva da su T i T^{-1} diferencijabilne, transportni uvjet

$$\mu(T^{-1}(A)) = \nu(A)$$

je preko teorema o zamjeni varijabli ekvivalentan s

$$f(x) = g(T(x)) |\det(\nabla T(x))|.$$

Dokaz se može pronaći u [2].

Glavni nedostatak ove formulacije je to što se čestice u točki x ne mogu dijeliti i transportirati na različite lokacije. Drugim riječima, funkcijom T nije moguće opisati dijeljenje mase.

2.2. Kantorovicheva formulacija

Kantorovicheva formulacija bavi se istim problemom optimalnog transporta, ali generalizira koncept transportne funkcije tako da se dopušta dijeljenje mase. I dalje imamo $\mu \in \mathcal{P}(X)$, $\nu \in \mathcal{P}(Y)$ te funkciju troška $c : X \times Y \rightarrow [0, +\infty]$. Uvodimo transportni plan $\pi \in \Pi(\mu, \nu)$, gdje je $\Pi(\mu, \nu)$ skup transportnih planova s μ na ν . Možemo na to gledati na način da $d\pi(x, y)$ predstavlja količinu mase koja se transportira s x na y . Na ovaj smo način generalizirali transportno preslikavanje T koje smo uveli kod Mongeove formulacije i koje nije dopuštalo dijeljenje mase.

Zahtijevamo da su zadovoljeni sljedeći uvjeti: $\pi(A \times Y) = \mu(A)$, $\pi(X \times B) = \nu(B)$ za sve izmjerive skupove $A \subseteq X$, $B \subseteq Y$, odnosno da su μ i ν marginalne distribucije vjerojatnosne mjere π . To nam garantira da je transportirana masa očuvana.

Sada Kantorovichevu formulaciju (detaljnije u [3]) možemo zapisati kao

$$\mathcal{L}_c(\mu, \nu) = \inf_{\pi} \left\{ \int c(x, y) d\pi(x, y) : \pi \in \Pi(\mu, \nu) \right\}. \quad (2.2)$$

Uzmimo sada $X = Y = \mathbb{R}^d$, a za funkciju troška uzimamo $c(x, y) = \|x - y\|^2$, gdje su $x, y \in \mathbb{R}^d$. Neka su nam $\mu, \nu \in \mathcal{P}(\mathbb{R}^d)$. Kantorovicheva nam formulacija daje

$$\mathcal{L}(\mu, \nu) = \inf_{\pi} \left\{ \int \|x - y\|^2 d\pi(x, y) : \pi \in \Pi(\mu, \nu) \right\}.$$

Pokazuje se da $\sqrt{\mathcal{L}}$ zadovoljava svojstva metrike [3] na prostoru vjerojatnosnih mjera $\mathcal{P}(\mathbb{R}^d)$. Ta se udaljenost, u ovom slučaju, naziva L^2 -Wassersteinova udaljenost i označava se

$$\mathcal{W}_2(\mu, \nu) := \inf_{\pi} \left\{ \int \|x - y\|^2 d\pi(x, y) : \pi \in \Pi(\mu, \nu) \right\}^{\frac{1}{2}}.$$

Navedimo sada formalnu definiciju.

Definicija 1 (L^p -Wassersteinova udaljenost) Za $\mu, \nu \in \mathcal{P}_p(\Omega)$ i $p \in [1, \infty)$ izraz

$$\mathcal{W}_p(\mu, \nu) = \inf_{\pi} \left\{ \int_{\Omega \times \Omega} \|x - y\|^p d\pi : \pi \in \Pi(\mu, \nu) \right\}^{\frac{1}{p}}$$

nazivamo L^p -Wassersteinova udaljenost između μ i ν . To je p -ti korijen minimalne transportne cijene za funkciju troška $c(x, y) = \|x - y\|^p$.

Ovako definirana udaljenost $\mathcal{W}_p$ zadovoljava svojstva metričke [3]. Metrički prostor vjerojatnosnih mjera $(\mathcal{P}_p(\Omega), \mathcal{W}_p)$, induciran ovom metrikom, naziva se Wassersteinovim prostorom.

2.3. Dinamička formulacija

U ostatku rada pretpostavljamo da su vjerojatnosne mjere μ_0, μ_1 apsolutno neprekidne s obzirom na Lebesgueovu mjeru, odnosno imaju gustoće $f_0, f_1 : \mathbb{R}^d \rightarrow [0, \infty)$, koja se još naziva Radon-Nikodymova derivacija $f_0 = d\mu_0/dx$.

Motivaciju za uvođenje dinamičke formulacije optimalnog transporta nalazimo u dinamici fluida. Nalazimo se u $\mathbb{R}^d$ i pretpostavljamo da u trenutku $t = 0$ gustoću fluida možemo opisati s $f_0 : \mathbb{R}^d \rightarrow [0, \infty)$. Taj fluid se giba i u trenutku $t = 1$ poprima gustoću $f_1 : \mathbb{R}^d \rightarrow [0, \infty)$. S $f_t : \mathbb{R}^d \times [0, 1] \rightarrow [0, \infty)$ ćemo označiti gustoću fluida u trenutku $t \in [0, 1]$. Napomenimo da $t \in [0, 1]$ nije fizikalno vrijeme, već interpolacijski parametar između gustoća.

Pretpostavimo da brzinu čestica u trenutku t možemo opisati vektorskim poljem $v_t : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$. Cilj nam je minimizirati ukupnu kinetičku energiju, odnosno pronaći v i f koji zadovoljavaju idući minimizacijski problem

$$\min_{(v, f)} \int_0^1 \int_{\mathbb{R}^d} \frac{1}{2} \|v_t(x)\|^2 f_t(x) dx dt \quad (2.3)$$

i koji zadovoljavaju jednadžbu očuvanja mase uz početni i završni uvjet

$$\begin{cases} \frac{\partial f_t}{\partial t} + \operatorname{div}(f_t v_t) = 0 \\ f_{t=0} = f_0, f_{t=1} = f_1. \end{cases} \quad (2.4)$$

Benamou i Brenier su u radu [5] dokazali idući važan teorem.

Teorem 1 *Neka su f_0, f_1 početna i konačna gustoća te $f : \mathbb{R}^d \times [0, 1] \rightarrow [0, \infty)$ i $v : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ takvi da je zadovoljen uvjet (2.4). Tada je (2.3) jednak kvadratu L^2 -Wassersteinove udaljenosti između f_0 i f_1 , tj. vrijedi formula*

$$\mathcal{W}_2^2(f_0, f_1) = \min_{(v, \mu) \text{ uz (2.4)}} \int_0^1 \int_{\mathbb{R}^d} \frac{1}{2} \|v_t(x)\|^2 f_t(x) dx dt. \quad (2.5)$$

Ovaj nam teorem govori da su Kantorovicheva i dinamička formulacija ekvivalente i da možemo odabrati onu koja najbolje odgovara problemu koji rješavamo.

Jednom kada smo odredili optimalno vektorsko polje v , moguće je rekonstruirati optimalno transportni preslikavanje T . Za dani $x \in \mathbb{R}^d$, definiramo $t \mapsto T_t(x)$ rješavanjem

$$T_0(x) = x \quad i \quad \forall t > 0, \quad \frac{\partial T_t(x)}{\partial t} = v_t(T_t(x)).$$

Optimalno transportno preslikavanje zatim je dano s $T = T_1$.

Uvodimo zamjenu varijabli $(v, f) \mapsto (m, f)$, gdje m predstavlja količinu gibanja (moment) $m = fv$. Dobivamo konveksni optimizacijski problem (što su pokazali Benamou i Brenier [5]) na paru (m, f)

$$\min_{(m, f) \in C} \mathcal{J}(m, f) = \min_{(m, f) \in C} \int_0^1 \int_{\mathbb{R}^d} J(m(x, t), f(x, t)) dx dt, \quad (2.6)$$

gdje je

$$\forall (m, f) \in \mathbb{R}^d \times \mathbb{R}, \quad J(m, f) = \begin{cases} \frac{\|m\|^2}{2f}, & f > 0, \\ 0, & (m, f) = (0, 0), \\ +\infty, & \text{inače.} \end{cases} \quad (2.7)$$

i skup linearnih uvjeta

$$C = \{ (m, f) : \frac{\partial f}{\partial t} + \text{div}(m) = 0, f(\cdot, 0) = f_0, f(\cdot, 1) = f_1 \}. \quad (2.8)$$

Ključno je zahtijevati da moment $m_t(x)$ iščezava kada je $f_t(x) = 0$. To možemo opravdati

time što ako u nekoj točki nema mase, tamo ne može postojati impuls jer nema mase koja bi se kretala.

3. Diskretizacija

U ovome ćemo poglavlju diskretizirati relevantne funkcije i ograničenja naših problema. Kako bismo si olakšali posao, u nastavku ćemo se ograničiti na $X = Y = [0, 1]^2$, a svaka druga pravokutna mreža lako se dobije odgovarajućim skaliranjem. Sljedeće oznake, pojmove i metode koristimo iz [11].

3.1. Centrirana mreža

Mreža na kojoj aproksimiramo $\mathbf{m}$ i f dobivena je dijeljenjem $[0, 1]^3$ na $(N + 1) \times (P + 1) \times (Q + 1)$ točaka. Time dobivamo centriranu mrežu

$$\mathcal{G}_c = \left\{ \left(x_i = \frac{i}{N}, y_j = \frac{j}{P}, t_k = \frac{k}{Q} \right) \in [0, 1]^3 : (i, j, k) \in D_C \right\}.$$

Neka je $D_C = [N + 1] \times [P + 1] \times [Q + 1]$ skup indeksa $(i, j, k) \in D_C$ na centriranoj mreži, gdje je za svaki $n \in \mathbb{N}$, $[n] := \{0, 1, \dots, n - 1\}$. S $\mathcal{E}_c = (\mathbb{R}^3)^{D_C}$ ćemo označiti konačno dimenzionalan prostor centriranih varijabli, a naše ćemo diskretizirane varijable definirati s

$$V = (\mathbf{m}, f) \in \mathcal{E}_c = (\mathbf{m}_{i,j,k}, \mathbf{f}_{i,j,k}),$$

gdje je $\mathbf{m} \in (\mathbb{R}^2)^{D_C}$, $\mathbf{f} \in \mathbb{R}^{D_C}$ i $(i, j, k) \in D_C$.

3.2. Pomaknuta mreža

Pomaknute mreže (eng. staggered grids) koristimo jer nam omogućuju preciznu evaluaciju parcijalnih derivacija u točkama Kartezijevog sustava koristeći standardne centralne konačne razlike. Takav se pristup pokazao uspješnim u diskretizaciji parcijalnih diferencijalnih jednadžbi kod dinamike nestlačivih fluida [11]. Napominjemo da zbog toga što radimo s dvodi-

menzionalnim gustoćama, moramo imati tri pomaknute mreže, dvije za prostorne koordinate i jednu za vremensku. Neka su

$$D_S^x = [N + 2] \times [P + 1] \times [Q + 1]$$

$$D_S^y = [N + 1] \times [P + 2] \times [Q + 1]$$

$$D_S^t = [N + 1] \times [P + 1] \times [Q + 2]$$

Našu pomaknutu mrežu definiramo sa

$$\mathcal{G}_S^x = \left\{ \left(x_i = \frac{i - \frac{1}{2}}{N}, y_j = \frac{j}{P}, t_k = \frac{k}{Q} \right) \in \left[\frac{-1}{2N}, 1 + \frac{1}{2N} \right] \times [0, 1]^2 : (i, j, k) \in D_S^x \right\},$$

$$\mathcal{G}_S^y = \left\{ \left(x_i = \frac{i}{N}, y_j = \frac{j - \frac{1}{2}}{P}, t_k = \frac{k}{Q} \right) \in [0, 1] \times \left[\frac{-1}{2P}, 1 + \frac{1}{2P} \right] \times [0, 1] : (i, j, k) \in D_S^y \right\},$$

$$\mathcal{G}_S^z = \left\{ \left(x_i = \frac{i}{N}, y_j = \frac{j}{P}, t_k = \frac{k - \frac{1}{2}}{Q} \right) \in [0, 1]^2 \times \left[\frac{-1}{2Q}, 1 + \frac{1}{2Q} \right] : (i, j, k) \in D_S^t \right\}.$$

Varijable diskretizirane na pomaknutoj mreži označavamo sa

$$U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_S,$$

gdje su $\mathbf{m}^x \in \mathbb{R}^{D_S^x}$, $\mathbf{m}^y \in \mathbb{R}^{D_S^y}$, $\mathbf{f}^t \in \mathbb{R}^{D_S^t}$, a $\mathcal{E}_S = \mathbb{R}^{D_S^x} \times \mathbb{R}^{D_S^y} \times \mathbb{R}^{D_S^t}$ konačno dimenzionalan prostor pomaknutih varijabli.

3.3. Pomoćne definicije

Svi objekti s kojima radimo su konačnodimenzionalni vektori realnih brojeva indeksirani mrežnim točkama, pa pripadaju Hilbertovom prostoru sa standardnim skalarnim produktom. Prvo ćemo definirati pripadne skalarne produkte.

Definicija 2 (Skalarni produkt na $\mathcal{E}_c$) $\langle \cdot, \cdot \rangle_C : \mathcal{E}_c \times \mathcal{E}_c \rightarrow \mathbb{R}$, za proizvoljne $V = (\mathbf{m}, \mathbf{f}) \in \mathcal{E}_c$ i $\tilde{V} = (\tilde{\mathbf{m}}, \tilde{\mathbf{f}}) \in \mathcal{E}_c$ definiramo sa

$$\langle V, \tilde{V} \rangle_C = \sum_{(i,j,k) \in D_c} (\langle \mathbf{m}_{i,j,k} + \tilde{\mathbf{m}}_{i,j,k} \rangle + \mathbf{f}_{i,j,k} \tilde{\mathbf{f}}_{i,j,k}).$$

Definicija 3 (Skalarni produkt na $\mathcal{E}_S$) $\langle \cdot, \cdot \rangle_S : \mathcal{E}_S \times \mathcal{E}_S \rightarrow \mathbb{R}$ definiramo sa

$$\langle U, \tilde{U} \rangle_S = \sum_{(i,j,k) \in D_S^x} m_{i,j,k}^x \tilde{m}_{i,j,k}^x + \sum_{(i,j,k) \in D_S^y} m_{i,j,k}^y \tilde{m}_{i,j,k}^y + \sum_{(i,j,k) \in D_S^t} f_{i,j,k}^t \tilde{f}_{i,j,k}^t,$$

za proizvoljne $U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_S$ i $\tilde{U} = (\tilde{\mathbf{m}}^x, \tilde{\mathbf{m}}^y, \tilde{\mathbf{f}}^t) \in \mathcal{E}_S$

Označimo s $H = \mathcal{E}_S \times \mathcal{E}_C$ zajednički vektorski prostor s pripadnim elementom $Z = (U, V)$.

Definicija 4 (Skalarni produkt na H) $\langle \cdot, \cdot \rangle_H : H \times H \rightarrow \mathbb{R}$ definiramo sa

$$\langle Z, \tilde{Z} \rangle_H = \langle (U, V), (\tilde{U}, \tilde{V}) \rangle_H = \langle U, \tilde{U} \rangle_S + \langle V, \tilde{V} \rangle_C,$$

za proizvoljne $Z, \tilde{Z} \in H$.

Navedimo sada definicije nekih važnih operatora koje ćemo dalje koristiti.

Definicija 5 (Operator interpolacije) Operator interpolacije $\mathcal{I} : \mathcal{E}_S \rightarrow \mathcal{E}_C$ definiramo po komponentama

$$\mathcal{I}(U)_{i,j,k} := \left(\frac{m_{i,j,k}^x + m_{i+1,j,k}^x}{2}, \frac{m_{i,j,k}^y + m_{i,j+1,k}^y}{2}, \frac{f_{i,j,k}^t + f_{i,j,k+1}^t}{2} \right) \quad (3.1)$$

za sve $(i, j, k) \in D_C$ i sve $U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_S$.

Definicija 6 (Operator divergencije) Prostorno-vremenski operator divergencije

$\text{div} : \mathcal{E}_S \rightarrow \mathbb{R}^{D_C}$ definiran je po komponentama s

$$\text{div}(U)_{i,j,k} = N (\bar{m}_{i+1,j,k}^x - \bar{m}_{i,j,k}^x) + P (\bar{m}_{i,j+1,k}^y - \bar{m}_{i,j,k}^y) + Q (\bar{f}_{i,j,k+1}^t - \bar{f}_{i,j,k}^t) \quad (3.2)$$

za sve $(i, j, k) \in D_C$ i za sve $U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_S$.

Neka su $\mathbf{f}_0, \mathbf{f}_1 \in \mathbb{R}^{(N+1) \times (P+1)}$ diskretizirana konačna i početna gustoća, a $U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_S$. Osim rubnog uvjeta na gustoću

$$\mathbf{f}_{i,j,0}^t = \mathbf{f}_0, \quad \mathbf{f}_{i,j,Q+1}^t = \mathbf{f}_1, \quad \forall (i, j) \in [N+1] \times [P+1]$$

namećemo i rubne uvjete na moment $\mathbf{m}^x \in \mathbb{R}^{D_s^x}$, $\mathbf{m}^y \in \mathbb{R}^{D_s^y}$

$$\mathbf{m}_{0,j,k}^x = \mathbf{m}_{N+1,j,k}^x = 0, \quad \forall (j, k) \in [P+1] \times [Q+1]$$

$$\mathbf{m}_{i,0,k}^y = \mathbf{m}_{i,P+1,k}^y = 0, \quad \forall (i, k) \in [N+1] \times [Q+1],$$

odnosno zahtijevamo da na rubovima prostorne domene moment bude nula. Ovaj uvjet uvodimo jer ne želimo da masa "bježi" izvan domene. Uvedimo sada operator kojim ćemo dohvatiti vrijednosti na granicama pomaknute mreže.

Definicija 7 (Rubni operator) $b : \mathcal{E}_s \rightarrow \mathcal{B} := \mathbb{R}^{2(P+1)(Q+1)} \times \mathbb{R}^{2(N+1)(Q+1)} \times \mathbb{R}^{2(N+1)(P+1)}$ definiramo sa

$$b(U) = \left((\mathbf{m}_{0,j,k}^x, \mathbf{m}_{N+1,j,k}^x), (\mathbf{m}_{i,0,k}^y, \mathbf{m}_{i,P+1,k}^y), \mathbf{f}_{i,j,0}^t, \mathbf{f}_{i,j,Q+1}^t \right),$$

gdje su $(i, j, k) \in D_C$ te $U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_s$.

Ako uvedemo $b_0 = ((0, 0), (0, 0), \mathbf{f}_0, \mathbf{f}_1) \in \mathcal{B}$, rubne uvjete možemo zapisati kao

$$b(U) = b_0.$$

3.4. Diskretizacija kontinuiranog problema

Podsjetimo se kako izgleda konveksni optimizacijski problem koji smo dobili na kraju drugog poglavlja. Imamo

$$\min_{(\mathbf{m}, f) \in C} \mathcal{J}(\mathbf{m}, f) = \min_{(\mathbf{m}, f) \in C} \int_0^1 \int_{\mathbb{R}^d} J(\mathbf{m}(x, t), f(x, t)) \, dx dt,$$

gdje je

$$\forall (\mathbf{m}, f) \in \mathbb{R}^d \times \mathbb{R}, \quad J(\mathbf{m}, f) = \begin{cases} \frac{\|\mathbf{m}\|^2}{2f}, & f > 0, \\ 0, & (\mathbf{m}, f) = (0, 0), \\ +\infty, & \text{inače.} \end{cases}$$

i skup uvjeta

$$C = \{ (\mathbf{m}, f) : \frac{\partial f}{\partial t} + \operatorname{div}(\mathbf{m}) = 0, f(\cdot, 0) = f_0, f(\cdot, 1) = f_1 \}.$$

Sada ćemo taj problem diskretizirati korištenjem mreža i operatora koje smo definirali u ovom poglavlju. Diskretni funkcional $\mathcal{J}_d$ je za $V = (\mathbf{m}, f) \in \mathcal{E}_c$ dan s

$$\mathcal{J}_d(V) = \sum_{l \in D_C} J(\mathbf{m}_l, f_l). \quad (3.3)$$

Diskretni skup ograničenja C_d odgovara skupu linearnih uvjeta C , odnosno diskretiziranoj jednadžbi očuvanja mase s rubnim uvjetima

$$C_d = \{ U \in \mathcal{E}_s : \operatorname{div}(U) = 0, b(U) = b_0 \}.$$

Za zatvoreni konveksni skup C_d , definiramo pripadnu indikatorsku funkciju

$$\iota_{C_d}(U) = \begin{cases} 0, & U \in C_d, \\ +\infty, & \text{inače.} \end{cases}$$

Zbog toga što smo argument $V \in \mathcal{E}_c$ definirali preko interpolacijskog operatora $\mathcal{I}$ koristeći $U \in \mathcal{E}_s$, moramo zadovoljiti identitet $\mathcal{I}(U) = V$. Za to koristimo indikatorsku funkciju

$$\iota_{C_{c,s}}(U, V),$$

gdje je

$$C_{c,s} = \{ (U, V) \in H : V = \mathcal{I}(U) \}.$$

Početni kontinuirani problem (2.6) je aproksimiran na diskretnoj mreži rješavanjem sljedećeg konačno dimenzionalnog konveksnog problema

$$\min_{(U,V) \in H} \mathcal{J}_d(V) + \iota_{C_d}(U) + \iota_{C_{c,s}}(U, V), \quad (3.4)$$

gdje je $Z = (U, V) \in H$.

Sada kada smo diskretizirali naš problem možemo započeti s definiranjem potrebnih proksimalnih operatora.

4. Douglas-Rachfordov algoritam

Minimizacija konveksnog funkcionala $\mathcal{F}$ na nekom Hilbertovom prostoru H zahtjeva uporabu algoritama koji su prilagođeni karakteristikama tog funkcionala. Glatke funkcije minimiziraju se uporabom metoda gradijentnog spusta, gdje se iterativno primjenjuje $\text{Id}_H - \gamma \nabla \mathcal{F}$ s dovoljno malim korakom $\gamma > 0$. Takve se metode ne mogu koristiti za minimizaciju neglatkih funkcija kakvima se mi bavimo [12].

Metode proksimalnog cijepanja zamjenjuju gradijentni spust s proksimalnim operatorom.

Definicija 8 (Proksimalni operator) $\text{Prox}_{\gamma\mathcal{F}} : H \rightarrow H$, gdje je $\mathcal{F} : H \rightarrow \mathbb{R} \cup \{+\infty\}$ konveksna funkcija, dan je s

$$\text{Prox}_{\gamma\mathcal{F}}(z) = \underset{\tilde{z} \in H}{\operatorname{argmin}} \frac{1}{2} \|z - \tilde{z}\|^2 + \gamma \mathcal{F}(\tilde{z}). \quad (4.1)$$

Proksimalni operator $\text{Prox}_{\gamma\mathcal{F}}(z)$ za predani argument z daje točku koja daje kompromis između minimizacije $\mathcal{F}$ i blizine z . Kako bismo mogli koristiti proksimalne operatore u optimizacijskom problemu, njihovo računanje mora biti efikasno. Funkcional $\mathcal{F}$ takav da $\text{Prox}_{\gamma\mathcal{F}}$ ima zatvoreni oblik zovemo *jednostavnim*. Glavna ideja iza metoda proksimalnog cijepanja jest podijeliti funkcional $\mathcal{F}$ u sumu jednostavnih funkcionala. Postoje mnoge metode proksimalnog cijepanja, a u ovome radu baviti ćemo se Douglas-Rachfordovim (DR) algoritmom [12]. On nam omogućuje rješavanje problema oblika

$$\min_{z \in H} \mathcal{G}_1(z) + \mathcal{G}_2(z), \quad (4.2)$$

gdje su G_1 i G_2 dvije jednostavne konveksne funkcije definirane na nekom Hilbertovom prostoru H .

Iteracije DR algoritma definiraju niz $(z^{(l)}, w^{(l)}) \in H^2$ koristeći inicijalno $(z^0, w^0) \in H^2$ i

$$\begin{aligned} w^{(l+1)} &= w^{(l)} + \alpha(\text{Prox}_{\gamma\mathcal{G}_1}(2z^{(l)} - w^{(l)}) - z^{(l)}), \\ z^{(l+1)} &= \text{Prox}_{\gamma\mathcal{G}_2}(w^{(l+1)}), \end{aligned} \quad (4.3)$$

gdje su $\alpha, \gamma \in \mathbb{R}$. Za $0 < \alpha < 2$ i $\gamma > 0$, pokazuje se da niz iteracija $(z^{(l)})$ konvergira k rješenju z^* problema (4.2) [12].

Postoji više načina povezivanja našeg optimizacijskog problema s (4.3), što rezultira s četiri inačice algoritma. Pokazuje se da sve inačice imaju slične performanse pa ćemo se ograničiti na tzv. *asimetrični-DR* (A-DR) [11] uz korištenje konsenzusa [12]. Minimizaciju provodimo nad $Z = (Z^1, Z^2, Z^3) \in H^3$, biramo težine $\omega_1, \omega_2, \omega_3 > 0$ takve da $\omega_1 + \omega_2 + \omega_3 = 1$, a funkcionalne za minimizaciju postavljamo na

$$\mathcal{G}_1(Z) = \mathcal{J}_d(Z^1) + \iota_{C_d}(Z^2) + \iota_{C_{c,s}}(Z^3), \quad \mathcal{G}_2(Z) = \iota_D(Z),$$

gdje je D skup

$$D = \{(Z, Z, Z) : Z \in H\}.$$

Ideja je ta da nam $\iota_D(Z)$ osigurava da su $Z^1 = Z^2 = Z^3$. Razlog zašto se koristi metoda konsenzusa bit će jasnija na kraju ovog poglavlja gdje ćemo raspisati iteracije DR algoritma u terminima naših operatora.

Sada je potrebno odrediti proksimalne operatore koji su definirani iznad. Zato ćemo najprije navesti neka korisna svojstva proksimalnih operatora koja će nam olakšati posao.

Propozicija 1 (Separabilnost proksimalnog operatora) *Ako je funkcija f separabilna po dvije varijable, odnosno*

$$f(x, y) = \varphi(x) + \psi(y)$$

tada vrijedi

$$\text{Prox}_{\gamma f}(u, v) = (\text{Prox}_{\gamma\varphi}(u), \text{Prox}_{\gamma\psi}(v)).$$

Analogno, ako je f potpuno separabilna, odnosno

$$f(x) = \sum_{i=1}^n f_i(x_i),$$

onda vrijedi

$$(\text{Prox}_{\gamma f}(v))_i = \text{Prox}_{\gamma f_i}(v_i).$$

Dokaz ove tvrdnje može se pronaći u [12].

Kada je F_C indikatorska funkcija za skup C , proksimalni operator od F_C svodi se na euklidsku projekciju na C . Zato se proksimalni operatori često smatraju generalizacijom projekcija [12]. Ovo će postati očigledno kada će se računati relevantni proksimalni operatori kasnije.

Također, fiksna točka proksimalnog operatora od f je ujedno i minimizator od f . To znači da je $\text{Prox}_{\gamma f}(z^*) = z^*$ ako i samo ako z^* minimizira f . Zato se proksimalni operatori mogu interpretirati kao rješavanje optimizacijskih problema traženjem fiksnih točaka tih operatora [12].

4.1. Proksimalni operator od $\mathcal{J}_d$

U ovome poglavlju računamo proksimalni operator od funkcionala $\mathcal{J}_d$ definiranog u (3.3). Ovdje koristimo svojstvo separabilnosti i dobivamo (za proizvoljan $V \in \mathcal{E}_c$)

$$(\text{Prox}_{\gamma \mathcal{J}_d}(V))_{l \in D_C} = \text{Prox}_{\gamma J}(V_l)_{l \in D_C},$$

gdje je $l = (i, j, k) \in D_C$. Neka su $\tilde{\mathbf{m}} \in \mathbb{R}^2, \tilde{f} \in \mathbb{R}$ proizvoljni, ali fiksirani argumenti. Korištenjem (4.1) dobijemo

$$\begin{aligned} \text{Prox}_{\gamma J}(\tilde{\mathbf{m}}, \tilde{f}) &= \underset{(\mathbf{m}, f) \in \mathbb{R}^3}{\text{argmin}} \frac{1}{2} \|(\tilde{\mathbf{m}}, \tilde{f}) - (\mathbf{m}, f)\|^2 + \gamma J(\mathbf{m}, f) \\ &= \underset{(\mathbf{m}, f) \in \mathbb{R}^3}{\text{argmin}} \frac{1}{2} (\|\tilde{\mathbf{m}} - \mathbf{m}\|^2 + (\tilde{f} - f)^2) + \gamma \frac{\|\mathbf{m}\|^2}{2f}. \end{aligned}$$

Označimo s $K(\mathbf{m}, f) = \frac{1}{2} \|\mathbf{m} - \tilde{\mathbf{m}}\|^2 + \frac{1}{2} (f - \tilde{f})^2 + \gamma \frac{\|\mathbf{m}\|^2}{2f}$. S obzirom da je K glatka i strogo konveksna funkcija na $f > 0$, nužan i dovoljan uvjet za minimum je $\nabla K(\mathbf{m}, f) = \mathbf{0}$. Računanjem gradijenta i izjednačavanjem s nulom dobijemo

$$f - \tilde{f} - \gamma \frac{\|\mathbf{m}\|^2}{2f^2} = 0$$

i

$$\mathbf{m} - \tilde{\mathbf{m}} + \gamma \frac{\mathbf{m}}{f} = \mathbf{0}.$$

Vrijednost $\mathbf{m}$ dobivamo formulom

$$\mathbf{m} = \frac{f\tilde{\mathbf{m}}}{f + \gamma},$$

a vrijednost f kao nultočku polinoma trećeg stupnja

$$f^3 + (2\gamma - \tilde{f})f^2 + (\gamma^2 - 2\gamma\tilde{f})f - \gamma^2\tilde{f} - \frac{\gamma}{2}\|\tilde{\mathbf{m}}\|^2 = 0.$$

S f^* označavamo traženu (najveću pozitivnu realnu) nultočku i, za sve $f^* \geq 0$, definiramo

$$\mu(f^*) = \frac{f^*\tilde{\mathbf{m}}}{f^* + \gamma}$$

i proksimalni operator

$$\text{Prox}_{\gamma J}(\tilde{\mathbf{m}}, \tilde{f}) = \begin{cases} (\mu(f^*), f^*), & f^* > 0 \\ (0, 0), & \text{inače} \end{cases} \quad (4.4)$$

4.2. Proksimalni operator od ι_{C_d}

Konveksni skup C_d može se zapisati kao afin skup

$$C_d = \{U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_s : AU = \mathbf{y}\},$$

gdje je $A : \mathcal{E}_s \rightarrow \mathbb{R}^{D_c} \times \mathcal{B}$ dan sa $A(U) = (\text{div}(U), b(U))$, a $\mathbf{y} = (\mathbf{0}, b_0) \in \mathbb{R}^{D_c} \times \mathcal{B}$. Naš proksimalni operator možemo zapisati kao (korištenjem (4.1))

$$\text{Prox}_{\gamma \iota_{C_d}}(\tilde{U}) = \underset{U \in C_d}{\text{argmin}} \frac{1}{2}\|\tilde{U} - U\|^2 + \gamma \iota_{C_d}(U).$$

Sada ćemo provesti minimizaciju, a kasnije ćemo dati argument koji minimizira. Neka je $\lambda \in \mathbb{R}^{D_c} \times \mathcal{B}$ proizvoljan. Korištenjem metode Lagrangeovih multiplikatora možemo gornji

problem uvjetne minimizacije zamijeniti bezuvjetnim problemom. Imamo

$$\begin{aligned}
& \min_{U \in \mathcal{C}_d} \frac{1}{2} \|\tilde{U} - U\|^2 + \gamma \iota_C(U) \\
&= \min_{U \in \mathcal{E}_s} \frac{1}{2} \|\tilde{U} - U\|^2 + \sup_{\lambda \in \mathbb{R}^{D_C} \times \mathcal{B}} \langle \lambda, AU - y \rangle \\
&= \sup_{\lambda \in \mathbb{R}^{D_C} \times \mathcal{B}} \min_{U \in \mathcal{E}_s} \frac{1}{2} \|\tilde{U} - U\|^2 + \langle A^* \lambda, U \rangle - \langle \lambda, y \rangle \\
&= \sup_{\lambda \in \mathbb{R}^{D_C} \times \mathcal{B}} \min_{U \in \mathcal{E}_s} \frac{1}{2} (\langle U, U \rangle - 2\langle \tilde{U}, U \rangle + \langle \tilde{U}, \tilde{U} \rangle) + \langle A^* \lambda, U \rangle - \langle \lambda, y \rangle,
\end{aligned}$$

gdje je $A^* : \mathbb{R}^{D_C} \times \mathcal{B} \rightarrow \mathcal{E}_s$ adjungirani operator od A . Izraz prvo deriviramo po U te dobijemo

$$\frac{1}{2}(2U^T - 2\tilde{U}^T) + (A^* \lambda)^T = U^T - \tilde{U}^T + (A^* \lambda)^T,$$

a nakon izjednačavanja s nulom imamo

$$U^T - \tilde{U}^T + (A^* \lambda)^T = 0 \implies U = \tilde{U} - A^* \lambda.$$

Novi izraz za U vraćamo u gornju minimizaciju i nastavljamo s računom

$$\begin{aligned}
& \sup_{\lambda \in \mathbb{R}^{D_C} \times \mathcal{B}} \frac{1}{2} \|\tilde{U} - \tilde{U} + A^* \lambda\|^2 + \langle A^* \lambda, \tilde{U} - A^* \lambda \rangle - \langle \lambda, y \rangle \\
&= \sup_{\lambda \in \mathbb{R}^{D_C} \times \mathcal{B}} \frac{1}{2} \|A^* \lambda\|^2 - \|A^* \lambda\|^2 + \langle A^* \lambda, \tilde{U} \rangle - \langle \lambda, y \rangle \\
&= \sup_{\lambda \in \mathbb{R}^{D_C} \times \mathcal{B}} -\frac{1}{2} \|A^* \lambda\|^2 + \langle \lambda, A\tilde{U} - y \rangle \\
&= \sup_{\lambda \in \mathbb{R}^{D_C} \times \mathcal{B}} -\frac{1}{2} \langle \lambda, AA^* \lambda \rangle + \langle \lambda, A\tilde{U} - y \rangle
\end{aligned}$$

Dobiveno deriviramo po λ

$$-\frac{1}{2}(\lambda^T AA^* + \lambda^T (AA^*)^T) + (A\tilde{U} - y)^T = -\lambda^T (AA^*) + (A\tilde{U} - y)^T$$

i izjednačavamo s nula

$$\lambda^T (AA^*) = (A\tilde{U} - y)^T \implies \lambda^T = (A\tilde{U} - y)^T (AA^*)^{-1} \implies \lambda = (AA^*)^{-1} (A\tilde{U} - y)$$

Dobiveni izraz za λ uvrštavamo u formulu za U

$$\begin{aligned} U &= \tilde{U} - A^*(AA^*)^{-1}(A\tilde{U} - y) \\ &= \tilde{U} - A^*(AA^*)^{-1}A\tilde{U} + A^*(AA^*)^{-1}y. \end{aligned}$$

Na kraju dobijemo ortogonalni projektor na konveksni skup C_d

$$\text{Proj}_{C_d}(\tilde{U}) = (\text{Id} - A^*(AA^*)^{-1}A)\tilde{U} + A^*(AA^*)^{-1}y.$$

U potpoglavlju 5.3.4. pokazujemo da se računanje ovog proksimalnog operatora svodi na rješavanje Laplaceove jednačbe s Neumannovim rubnim uvjetima korištenjem diskretne kosinusove transformacije.

4.3. Proksimalni operator od $\iota_{C_{c,s}}$

Proksimalni operator od $\iota_{C_{c,s}}(U, V)$ možemo dobiti reformulirajući skup $C_{c,s}$ s linearnom jednačbom $Bx = y$. Početni skup $C_{c,s}$ definiran je s

$$C_{c,s} = \{ Z = (U, V) \in H : I(U) = V \},$$

gdje je I operator interpolacije sa skupa pomaknutih mreža $\mathcal{E}_s$ na skup centralnih mreža $\mathcal{E}_c$. Stavljamo

$$C_{c,s} = \{ Z = (U, V) \in \mathcal{E}_s \times \mathcal{E}_c : BZ = y \},$$

gdje je $B : H \rightarrow \mathcal{E}_c$ dan s $B(Z) = I(U) - V$, a $y = 0 \in \mathcal{E}_c$. Nastavljamo analogno kao i za proksimalni operator indikatorske funkcije skupa C_d . Proksimalni operator koji računamo (ponovo dobiven korištenjem (4.1)) je

$$\text{Prox}_{\gamma\iota_{C_{c,s}}}(\tilde{Z}) = \underset{Z \in C_{c,s}}{\text{argmin}} \frac{1}{2} \|\tilde{Z} - Z\|^2 + \gamma\iota_{C_{c,s}}(Z).$$

Tražimo minimum, a poslije i argument koji minimizira. Neka je $\lambda \in \mathcal{E}_c$. Imamo

$$\begin{aligned}
& \min_{Z \in C_{c,s}} \frac{1}{2} \|\tilde{Z} - Z\|^2 + \gamma \iota_{C_{c,s}}(Z) \\
&= \min_{Z \in H} \frac{1}{2} \|\tilde{Z} - Z\|^2 + \sup_{\lambda \in \mathcal{E}_c} \langle \lambda, BZ \rangle \\
&= \sup_{\lambda \in \mathcal{E}_c} \min_{Z \in H} \frac{1}{2} \|\tilde{Z} - Z\|^2 + \langle B^* \lambda, Z \rangle \\
&= \sup_{\lambda \in \mathcal{E}_c} \min_{Z \in H} \frac{1}{2} (\langle Z, Z \rangle - 2\langle \tilde{Z}, Z \rangle + \langle \tilde{Z}, \tilde{Z} \rangle) + \langle B^* \lambda, Z \rangle,
\end{aligned}$$

gdje je $B^* : \mathcal{E}_c \rightarrow H$ adjungirani operator od B . Derivacijom po Z dobijemo

$$\frac{1}{2}(2Z^T - 2\tilde{Z}^T) + (B^* \lambda)^T = Z^T - \tilde{Z}^T + (B^* \lambda)^T$$

te to izjednačavamo s nulom

$$Z^T - \tilde{Z}^T + (B^* \lambda)^T = 0 \implies Z = \tilde{Z} - B^* \lambda.$$

Novi izraz za Z vraćamo u gornju minimizaciju i nastavljamo s računom

$$\begin{aligned}
& \sup_{\lambda \in \mathcal{E}_c} \frac{1}{2} \|\tilde{Z} - \tilde{Z} + B^* \lambda\|^2 + \langle B^* \lambda, \tilde{Z} - B^* \lambda \rangle \\
&= \sup_{\lambda \in \mathcal{E}_c} \frac{1}{2} \|B^* \lambda\|^2 - \|B^* \lambda\|^2 + \langle B^* \lambda, \tilde{Z} \rangle \\
&= \sup_{\lambda \in \mathcal{E}_c} -\frac{1}{2} \|B^* \lambda\|^2 + \langle \lambda, B\tilde{Z} \rangle \\
&= \sup_{\lambda \in \mathcal{E}_c} -\frac{1}{2} \langle \lambda, BB^* \lambda \rangle + \langle \lambda, B\tilde{Z} \rangle.
\end{aligned}$$

Dobiveno deriviramo po λ

$$-\frac{1}{2}(\lambda^T BB^* + \lambda^T (BB^*)^T) + (B\tilde{Z})^T = -\lambda^T (BB^*) + (B\tilde{Z})^T$$

i izjednačavamo s nula

$$\lambda^T (BB^*) = (B\tilde{Z})^T \implies \lambda^T = (B\tilde{Z})^T (BB^*)^{-1} \implies \lambda = (BB^*)^{-1} (B\tilde{Z}).$$

Dobiveni izraz za λ uvrštavamo u formulu za Z

$$\begin{aligned} Z &= \tilde{Z} - B^*(BB^*)^{-1}(B\tilde{Z}) \\ &= (\text{Id} - B^*(BB^*)^{-1}B) \tilde{Z}. \end{aligned}$$

Na kraju dobijemo ortogonalni projektor na skup $C_{c,s}$

$$\text{Proj}_{C_{c,s}}(\tilde{Z}) = (\text{Id} - B^*(BB^*)^{-1}B)\tilde{Z}.$$

4.4. Definicija iteracija

Sada kada smo definirali sve potrebne proksimalne operatore, možemo definirati niz koji aproksimira rješenje našeg problema. Prisjetimo se kako općenito izgleda Douglas-Rachfordov algoritam.

$$\begin{aligned} w^{(l+1)} &= w^{(l)} + \alpha(\text{Prox}_{\gamma\mathcal{G}_1}(2z^{(l)} - w^{(l)}) - z^{(l)}), \\ z^{(l+1)} &= \text{Prox}_{\gamma\mathcal{G}_2}(w^{(l+1)}). \end{aligned}$$

Kao što smo već naveli, u našem slučaju vrijedi, uz $Z = (Z^1, Z^2, Z^3) \in H^3$

$$\mathcal{G}_1(Z) = \mathcal{J}_d(Z^1) + \iota_{C_d}(Z^2) + \iota_{C_{c,s}}(Z^3)$$

te

$$\mathcal{G}_2(Z) = \iota_D(Z).$$

Primijetimo odmah da, zbog separabilnosti vrijedi

$$\text{Prox}_{\gamma\mathcal{G}_1}(Z) = \left(\text{Prox}_{\gamma\mathcal{J}_d}(Z^1), \text{Prox}_{\gamma\iota_{C_d}}(Z^2), \text{Prox}_{\gamma\iota_{C_{c,s}}}(Z^3) \right).$$

Također se jednostavnim računom dobije

$$\text{Prox}_{\gamma\iota_D}(\tilde{Z}^3) = (Z, Z, Z),$$

gdje je $Z = \omega_1 \tilde{Z}^1 + \omega_2 \tilde{Z}^2 + \omega_3 \tilde{Z}^3$.

Neka su sada $\mathbf{z}^{(l)} = (x_l, x_l, x_l) \in D$ i $\mathbf{w}^{(l)} = (w_l^1, w_l^2, w_l^3) \in H^3$. Zbog separabilnosti za proksimalni operator od $\mathcal{G}_1$ vrijedi

$$\text{Prox}_{\gamma\mathcal{G}_1}(2\mathbf{z}^{(l)} - \mathbf{w}^{(l)}) = \left(\text{Prox}_{\gamma\mathcal{J}_d}(2x_l - w_l^1), \text{Prox}_{\gamma\iota_{C_d}}(2x_l - w_l^2), \text{Prox}_{\gamma\iota_{C_{c,s}}}(2x_l - w_l^3) \right).$$

Napravimo zamjenu varijabli $z_l^i := 2x_l - w_l^i$, gdje je $i \in \{1, 2, 3\}$ i uvedimo pomoćne oznake $p_l^1, p_l^2, p_l^3, p_l \in H$ radi ljepšeg zapisa. Prvo imamo

$$\begin{aligned} p_l^1 &= \text{Prox}_{\gamma\mathcal{J}}(z_l^1), \\ p_l^2 &= \text{Prox}_{\gamma\iota_C}(z_l^2), \\ p_l^3 &= \text{Prox}_{\gamma\iota_{C_{c,s}}}(z_l^3), \\ p_l &= \omega_1 p_l^1 + \omega_2 p_l^2 + \omega_3 p_l^3 \end{aligned}$$

te

$$\begin{aligned} w_{l+1}^1 &= w_l^1 + \alpha(p_l^1 - x_l), \\ w_{l+1}^2 &= w_l^2 + \alpha(p_l^2 - x_l), \\ w_{l+1}^3 &= w_l^3 + \alpha(p_l^3 - x_l). \end{aligned}$$

Zatim imamo

$$x_{l+1} = \text{Prox}_{\gamma\iota_D}(w_{l+1}) = \sum_{i=1}^3 \omega_i w_{l+1}^i = \sum_{i=1}^3 \omega_i w_l^i + \alpha \left(\sum_{i=1}^3 \omega_i p_l^i - x_l \right) = (1 - \alpha)x_l + \alpha p_l.$$

Na kraju ostaje

$$z_{l+1}^i = 2x_{l+1} - w_{l+1}^i = 2[(1 - \alpha)x_l + \alpha p_l] - [(2x_l - z_l^i) + \alpha(p_l^i - x_l)] = z_l^i + \alpha(2p_l - x_l - p_l^i),$$

za $i = \{1, 2, 3\}$.

Napišimo to sada sve zajedno na jednom mjestu. Imamo $z_l^1, z_l^2, z_l^3, p_l^1, p_l^2, p_l^3, p, x_l \in H$ za sve $l \in \mathbb{N}$. Gornji generalni oblik iteracije Douglas-Rachfordovog algoritma sada smo

preoblikovali u sljedeći iteracijski postupak.

$$\begin{aligned}
p_l^1 &= \text{Prox}_{\gamma\mathcal{J}}(z_l^1), \\
p_l^2 &= \text{Prox}_{\gamma\iota_C}(z_l^2), \\
p_l^3 &= \text{Prox}_{\gamma\iota_{C_c,s}}(z_l^3), \\
p_l &= \omega_1 \cdot p_l^1 + \omega_2 \cdot p_l^2 + \omega_3 \cdot p_l^3, \\
z_{l+1}^1 &= z_l^1 + \alpha \cdot (2p_l - x_l - p_l^1), \\
z_{l+1}^2 &= z_l^2 + \alpha \cdot (2p_l - x_l - p_l^2), \\
z_{l+1}^3 &= z_l^3 + \alpha \cdot (2p_l - x_l - p_l^3), \\
x_{l+1} &= (1 - \alpha) \cdot x_l + \alpha \cdot p_l.
\end{aligned} \tag{4.5}$$

U implementaciji uzimamo $\omega_1 = \omega_2 = \omega_3 = 1/3$, $\alpha = 1/75$ te $\gamma = 1.9$. Te su se brojke empirijski pokazale najboljima.

Ako je $\alpha \in \langle 0, 2 \rangle$, $\gamma > 0$ te $\omega_1, \omega_2, \omega_3 > 0$ takvi da $\omega_1 + \omega_2 + \omega_3 = 1$, onda $x_n \in H$ konvergira prema rješenju našeg diskretiziranog minimizacijskog problema. Dokaz se može pronaći ovdje [12]. Algoritam zaustavljamo ako $\|x_{l+1} - x_l\| < \epsilon$, gdje je ϵ zadana tolerancija ili ako se prekorači zadani broj iteracija.

5. Implementacija

S obzirom da je gornje opisani algoritam pogodan za masovnu paralelizaciju, prirodan jezik za implementaciju je CUDA C++. CUDA je platforma za paralelno računanje koju je razvila Nvidia. Omogućuje korisnicima da koristeći jezike poput C, C++, Fortran, Python i sl. pišu kod koji će se izvršavati na grafičkom procesoru (GPU). To je pogotovo korisno u slučajevima kada se izvode jednostavne numeričke operacije, ali u golemim količinama (iscrtavanje grafike, obrađivanje slika i videozapisa, znanstvene simulacije...) U nastavku ovog poglavlja detaljno ćemo opisati implementaciju, a nakon toga rezultate numeričkih simulacija.

5.1. Centrirana i pomaknuta mreža

Dvodimenzionalni prostorni kvadrat $[0, 1]^2$ diskretiziran je s parametrima N, P , dok je vremenski interval $[0, 1]$ diskretiziran s parametrom Q . Gornji algoritam zahtijeva korištenje centrirane $V \in \mathcal{E}_c$ i pomaknute $U \in \mathcal{E}_s$ mreže. S obzirom da je gustoća f skalarno polje, a moment m dvodimenzionalno vektorsko polje, za svaku je mrežu potrebno alocirati tri polja, jedno za gustoću i dva za moment (x komponenta i y komponenta). Tablica 5.1. daje pregled alociranih polja i njihovih dimenzija.

Pomaknuta mreža sastoji se od dva pomaknuta polja za moment i jednog pomaknutog za gustoću, odnosno $U = (mx, my, f_s)$. Analogno, centrirana se mreža sastoji od pripadnih centriranih polja $V = (mx_c, my_c, f_c)$. S obzirom da Douglas-Rachfordov algoritam treba

Tablica 5.1. Struktura polja.

Polje	Dimenzije $[x, y, t]$	Fizikalno značenje
mx	$(N + 2) \times (P + 1) \times (Q + 1)$	x -moment, pomaknut samo u x -osi
my	$(N + 1) \times (P + 2) \times (Q + 1)$	y -moment, pomaknut samo u y -osi
f_s	$(N + 1) \times (P + 1) \times (Q + 2)$	gustoća, pomaknuta samo u t -osi
mx_c, my_c, f_c	$(N + 1) \times (P + 1) \times (Q + 1)$	trojka V , sve na centriranoj mreži

nekoliko kopija para (U, V) na kojima će izvoditi jednostavne algebarske operacije preko cijelog para odjednom, korisno je tretirati $(U, V) \in \mathcal{E}_s \times \mathcal{E}_c$ kao jedan veliki vektor. Zato se umjesto šest zasebnih `thrust::device_vector` polja, struktura `JointLayout` spaja sva gore definirana polja u jedno ravno polje i samo pamti bajtni pomak. Na taj način operacije koje trebaju pristupiti cijelom paru mogu to jednostavno učiniti, a istovremeno proksimalni operatori kojima je bitna struktura mreže mogu pristupiti pojedinim pokazivačima na njima bitnim dijelovima polja.

```
struct JointLayout {
    GridDims g;
    long long off_mx_s, off_my_s, off_f_s, off_mx_c,
                off_my_c, off_f_c, total;
    double* mx_s(double* base) const { return base + off_mx_s; }
    ...
};
```

5.2. Implementacija proksimalnog operatora od $\mathcal{J}_d$

5.2.1. Rješavanje kubne jednadžbe

Ovaj se proksimalni operator svodi na traženje pozitivnog realnog korijena sljedeće kubne jednadžbe:

$$f^3 + (2\gamma - \tilde{f})f^2 + (\gamma^2 - 2\gamma\tilde{f})f - \gamma^2\tilde{f} - \frac{\gamma}{2}\|\tilde{\mathbf{m}}\|^2 = 0,$$

gdje je $\tilde{f} \in \mathbb{R}$ te $\tilde{\mathbf{m}} \in \mathbb{R}^2$.

Prema Descartesovom pravilu predznaka, uz $\gamma > 0$, ova kubna jednadžba ima jedinstven pozitivan realan korijen koji je traženo rješenje. Ako on ne postoji, za rješenje uzimamo $(f, \mathbf{m}) = (0, 0)$. Ova se jednadžba u kodu rješava primjenom Cardanove formule. Izdvojimo prvo koeficijente radi lakšeg zapisa:

$$a = 1, \quad b = 2\gamma - \tilde{f}, \quad c = \gamma^2 - 2\gamma\tilde{f}, \quad d = -\gamma^2\tilde{f} - \frac{\gamma}{2}\|\tilde{\mathbf{m}}\|^2.$$

Zatim uvrštavamo $f = t - b/3$ kako bi se eliminirao kvadratni član. Dobivamo $t^3 + pt + q = 0$, gdje su $p = c - b^2/3$ te $q = 2b^3/27 - bc/3 + d$. Nakon toga računamo diskriminantu

$\Delta = (\frac{q}{2})^2 + (\frac{p}{3})^3$. Ako je $\Delta > 0$ imamo slučaj jednog realnog korijena koji je jednak

$$t = \sqrt[3]{-\frac{q}{2} + \sqrt{\Delta}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\Delta}}.$$

Ako je $\Delta \leq 0$ imamo slučaj tri realna korijena dana s

$$t_k = 2\sqrt{-\frac{p}{3}} \cos\left(\frac{1}{3} \arccos(A) - \frac{2\pi k}{3}\right), \quad k = 0, 1, 2,$$

gdje je $A = \frac{3q}{2p} \sqrt{\frac{-3}{p}}$, što ograničavamo na $[-1, 1]$ ako bismo spriječili pogrešku zaokruživanja koja bi mogla argument funkcije arccos gurnuti izvan domene. U degeneriranom slučaju gdje je $\Delta \approx 0$ uzimamo $t = \sqrt[3]{-q}$.

5.2.2. Newtonova metoda

Svaki kandidat za korijen zatim prolazi kroz tri Newtonove iteracije. Te su iteracije računski jeftine, a povećavaju preciznost računa. Na kraju uzimamo najveći pozitivni korijen, a ako niti jedan nije pozitivan, vraćamo nulu. U nastavku je prikazan kod kojim je implementirana Newtonova metoda.

```
__device__ inline double cubic_eval(double f, double b,
                                   double c, double d) {
    return ((f + b) * f + c) * f + d;
}

__device__ inline double cubic_deriv(double f, double b, double c) {
    return (3.0 * f + 2.0 * b) * f + c;
}

__device__ inline double newton_polish(double f, double b,
                                       double c, double d) {
#pragma unroll
    for (int it = 0; it < 3; ++it) {
        double fp = cubic_deriv(f, b, c);
        if (fabs(fp) < 1e-14) break;
        f -= cubic_eval(f, b, c, d) / fp;
    }
}
```

```

    return f;
}

```

Kubni se polinom, te njegova derivacija, evaluiraju Hornerovom metodom u inline funkcijama pošto su potrebne operacije jeftine. U implementaciji Newtonove metode, s obzirom da imamo samo tri iteracije, glavna se petlja ”otpetljava” prevoditeljskom uputom `#pragma unroll`.

Kao što je u izvodu ovog proksimalnog operatora iskorišteno, on je u potpunosti separabilan i zato je odličan kandidat za masovnu paralelizaciju. U svakoj se točki $(i, j, k) \in D_C$ centrirane mreže ovaj račun može provesti paralelno.

5.3. Implementacija proksimalnog operatora od ι_{C_d}

5.3.1. Izvod izraza za AA^*

Prisjetimo se prvo dobivenog izraza za ovaj proksimalni operator. Njegova je uloga osigurati da su zadovoljeni jednadžba kontinuiteta i traženi rubni uvjeti. Za $\tilde{U} \in \mathcal{E}_s$, $A = (\text{div}, b)$ te $y = (0, b_0) \in \mathbb{R}^{D_C} \times \mathcal{B}$, gdje je $b_0 = ((0, 0), (0, 0), f_0, f_1) \in \mathcal{B}$ imamo

$$\text{Proj}_{C_d}(\tilde{U}) = \tilde{U} - A^*(AA^*)^{-1}(A\tilde{U} - y) \in \mathcal{E}_s.$$

S obzirom da operator b doslovno čita određene vrijednosti na rubovima pomaknute mreže, ovaj operator možemo preformulirati. Prvo uvodimo operator $M : \mathcal{E}_s \rightarrow \mathcal{E}_s$ koji poništava točno te vrijednosti na rubovima kojima b pristupa. Za proizvoljnu točku (i, j, k) na pomaknutoj mreži imamo

$$(MU)_{i,j,k} = \begin{cases} U_{i,j,k}, & (i, j, k) \text{ unutarnja,} \\ 0, & (i, j, k) \text{ rubna.} \end{cases}$$

Točka (i, j, k) rubna je ako indeks i poprimi vrijednost 0 ili $N + 1$, ili ako indeks j poprimi vrijednost 0 ili $P + 1$, ili ako indeks k poprimi vrijednost 0 ili $Q + 1$. Iz definicije M lako se vidi da je to idempotentni $M^2 = M$ te samoadjungirani operator $M = M^*$. Zato se radi o ortogonalnom projektoru pa svaki $U \in \mathcal{E}_s$ možemo zapisati kao

$$U = MU + (\text{Id} - M)U.$$

Definirajmo sada $\tilde{U}_{cl} \in \mathcal{E}_s$ tako da zadrži unutarnji dio od $\tilde{U}$, a rubovi se prepisu s b_0 . Neka je $B_0 \in \mathcal{E}_s$ takva da na rubovima ima upravo b_0 . Sada imamo

$$\tilde{U}_{cl} = M\tilde{U} + (\text{Id} - M)B_0.$$

Svaki $U \in \mathcal{E}_s$ za koji vrijedi $b(U) = b_0$ sada možemo zapisati kao $U = \tilde{U}_{cl} + D$, gdje je $D \in \mathcal{E}_s$ takav da na rubovima ima vrijednost nula, odnosno $D \in \text{Im}M$. Sada imamo

$$\|\tilde{U} - U\|^2 = \|(\tilde{U} - \tilde{U}_{cl}) - D\|^2.$$

Budući da se $\tilde{U}$ i $\tilde{U}_{cl}$ podudaraju u unutarnjim točkama, imamo $(\tilde{U} - \tilde{U}_{cl}) \perp \text{Im}M$, pa po Pitagorinom teoremu vrijedi

$$\|\tilde{U} - U\|^2 = \|\tilde{U} - \tilde{U}_{cl} - D\|^2 = \|\tilde{U} - \tilde{U}_{cl}\|^2 + \|D\|^2.$$

Kako član $\|\tilde{U} - \tilde{U}_{cl}\|^2$ ne ovisi o D , minimizacija greške $\|\tilde{U} - U\|^2$ svodi se na uvjetni problem minimalne norme unutar potprostora $\text{Im}M$:

$$\min_{D \in \text{Im}M} \frac{1}{2} \|D\|^2, \quad \text{uz uvjet } \text{div}(D) = -\text{div}(\tilde{U}_{cl}). \quad (5.1)$$

Taj uvjet dolazi iz zahtjeva da je $\text{div}(U) = \text{div}(\tilde{U}_{cl} + D) = 0$. Neka je $\text{div}|_{\text{Im}M} : \text{Im}M \rightarrow \mathbb{R}^{Dc}$ restrikcija operatora divergencije na $\text{Im}M$, a $\text{div}^*|_{\text{Im}M} : \mathbb{R}^{Dc} \rightarrow \text{Im}M$ njegov adjungirani operator. Tada je $\text{div}(D) = \text{div}|_{\text{Im}M}(D)$ s obzirom da je $D \in \text{Im}M$. Za sve $U \in \text{Im}M$ i sve $V \in \mathbb{R}^{Dc}$ mora vrijediti

$$\langle \text{div}|_{\text{Im}M} U, V \rangle = \langle U, \text{div}^*|_{\text{Im}M} V \rangle_{\text{Im}M} = \langle U, \text{div}^*|_{\text{Im}M} V \rangle_S,$$

gdje zadnja jednakost vrijedi jer je skalarni produkt na $\text{Im}M$ samo restrikcija onog na $\mathcal{E}_s$, a $U, \text{div}^*|_{\text{Im}M} V \in \text{Im}M \subseteq \mathcal{E}_s$. Dalje imamo

$$\langle \text{div}|_{\text{Im}M} U, V \rangle = \langle \text{div}(U), V \rangle = \langle U, \text{div}^*(V) \rangle_S.$$

Korištenjem $U = MU \in \text{Im}M$ i činjenice da je M samoadjungiran operator imamo

$$\langle U, \text{div}^*(V) \rangle_S = \langle MU, \text{div}^*(V) \rangle_S = \langle U, M(\text{div}^*(V)) \rangle_S.$$

Usporedbom dobivenih izraza zaključujemo da vrijedi $\operatorname{div}^*|_{\operatorname{Im}M}(V) = M(\operatorname{div}^*(V)) \in \operatorname{Im}M$ za $V \in \mathbb{R}^{D_C}$. Sada ćemo provesti minimizaciju norme. Neka je $\lambda \in \mathbb{R}^{D_C}$. Definirajmo najprije indikatorsku funkciju $\iota_{\operatorname{div}|_{\operatorname{Im}M}(D)=-\operatorname{div}(\tilde{U}_{cl})} : \operatorname{Im}M \rightarrow \{0, +\infty\}$

$$\iota_{\operatorname{div}|_{\operatorname{Im}M}(D)=-\operatorname{div}(\tilde{U}_{cl})}(D) = \begin{cases} 0, & \operatorname{div}|_{\operatorname{Im}M}(D) = -\operatorname{div}(\tilde{U}_{cl}), \\ +\infty, & \text{inače.} \end{cases}$$

Sada se uvjetni minimizacijski problem (5.1) može zapisati kao bezuvjetni problem:

$$\begin{aligned} & \min_{D \in \operatorname{Im}M} \frac{1}{2} \|D\|^2 + \iota_{\operatorname{div}|_{\operatorname{Im}M}(D)=-\operatorname{div}(\tilde{U}_{cl})}(D) \\ &= \min_{D \in \operatorname{Im}M} \frac{1}{2} \|D\|^2 + \sup_{\lambda \in \mathbb{R}^{D_C}} \langle \lambda, \operatorname{div}|_{\operatorname{Im}M} D + \operatorname{div}(\tilde{U}_{cl}) \rangle \\ &= \sup_{\lambda \in \mathbb{R}^{D_C}} \min_{D \in \operatorname{Im}M} \frac{1}{2} \|D\|^2 + \langle \lambda, \operatorname{div}|_{\operatorname{Im}M} D \rangle + \langle \lambda, \operatorname{div}(\tilde{U}_{cl}) \rangle. \end{aligned}$$

Fiksirajmo sada λ . S obzirom da je funkcija $D \mapsto \frac{1}{2} \|D\|^2 + \langle \lambda, \operatorname{div}|_{\operatorname{Im}M} D \rangle$ kvadratna na $\operatorname{Im}M$, nužan i dovoljan uvjet za minimum je da usmjerena derivacija u svakom smjeru $h \in \operatorname{Im}M$ iščezava. Imamo

$$\frac{d}{d\epsilon} \Big|_{\epsilon=0} \left(\frac{1}{2} \|D + \epsilon h\|^2 + \langle \lambda, \operatorname{div}|_{\operatorname{Im}M}(D + \epsilon h) \rangle \right) = \langle D, h \rangle_S + \langle \lambda, \operatorname{div}|_{\operatorname{Im}M} h \rangle = \langle D + \operatorname{div}^*|_{\operatorname{Im}M} \lambda, h \rangle_S,$$

odnosno mora vrijediti $D + \operatorname{div}^*|_{\operatorname{Im}M} \lambda \perp \operatorname{Im}M$, no kako su $D \in \operatorname{Im}M$ i $\operatorname{div}^*|_{\operatorname{Im}M} \lambda \in \operatorname{Im}M$, pa posljedično i njihov zbroj slijedi da je $D + \operatorname{div}^*|_{\operatorname{Im}M} \lambda = 0$, odnosno $D = -\operatorname{div}^*|_{\operatorname{Im}M} \lambda$. Uvrštavanjem u gornji izraz slijedi

$$g(\lambda) := \frac{1}{2} \|\operatorname{div}^*|_{\operatorname{Im}M} \lambda\|^2 - \langle \lambda, \operatorname{div}|_{\operatorname{Im}M}(\operatorname{div}^*|_{\operatorname{Im}M} \lambda) \rangle + \langle \lambda, \operatorname{div}(\tilde{U}_{cl}) \rangle.$$

Kako vrijedi $\langle \lambda, \operatorname{div}|_{\operatorname{Im}M}(\operatorname{div}^*|_{\operatorname{Im}M}(\lambda)) \rangle_C = \langle \operatorname{div}^*|_{\operatorname{Im}M} \lambda, A^* \lambda \rangle_S = \|\operatorname{div}^*|_{\operatorname{Im}M} \lambda\|^2$, sada imamo

$$g(\lambda) = \frac{1}{2} \|\operatorname{div}^*|_{\operatorname{Im}M} \lambda\|^2 - \|\operatorname{div}^*|_{\operatorname{Im}M} \lambda\|^2 + \langle \lambda, \operatorname{div}(\tilde{U}_{cl}) \rangle = -\frac{1}{2} \|\operatorname{div}^*|_{\operatorname{Im}M} \lambda\|^2 + \langle \lambda, \operatorname{div}(\tilde{U}_{cl}) \rangle.$$

Deriviranjem po λ i izjednačavanjem s nulom imamo

$$\frac{\partial}{\partial \lambda} g(\lambda) = -\operatorname{div}|_{\operatorname{Im}M}(\operatorname{div}^*|_{\operatorname{Im}M} \lambda) + \operatorname{div}(\tilde{U}_{cl}) = 0.$$

Iz toga slijedi $\lambda = (\operatorname{div}|_{\operatorname{Im} M} \operatorname{div}^*|_{\operatorname{Im} M})^{-1} \operatorname{div}(\tilde{U}_{cl})$, odnosno D koji minimiziram (5.1) je

$$\begin{aligned} D^* &= -\operatorname{div}^*|_{\operatorname{Im} M} \lambda \\ &= -\operatorname{div}^*|_{\operatorname{Im} M} (\operatorname{div}|_{\operatorname{Im} M} \operatorname{div}^*|_{\operatorname{Im} M})^{-1} \operatorname{div}(\tilde{U}_{cl}) \\ &= -M(\operatorname{div}^* ((\operatorname{div}|_{\operatorname{Im} M} \operatorname{div}^*|_{\operatorname{Im} M})^{-1} \operatorname{div}(\tilde{U}_{cl}))). \end{aligned}$$

Sada možemo zapisati izraz za rezultat izvođenja traženog proksimalnog operatora

$$U = \tilde{U}_{cl} + D^* = \tilde{U}_{cl} - M(\operatorname{div}^* ((\operatorname{div}|_{\operatorname{Im} M} \operatorname{div}^*|_{\operatorname{Im} M})^{-1} \operatorname{div}(\tilde{U}_{cl}))).$$

Neka je sada $U \in \mathcal{E}_s$ proizvoljan. Moramo odrediti eksplicitni izraz za $\operatorname{div}|_{\operatorname{Im} M} \operatorname{div}^*|_{\operatorname{Im} M}$. Fokusirajmo se prvo na x -os (fiksiramo indekse j, k), s $N + 1$ centriranih točaka i $N + 2$ pomaknutih točaka. Zbog čitljivosti pišemo U_i umjesto $U_{i,j,k}$. Za adjungirani operator div^* (za fiksne j, k) vrijedi

$$(\operatorname{div}^* U)_i = N U_{i-1} - N U_i,$$

gdje uz član U_{i-1} mora vrijediti $0 \leq i-1 \leq N$, a uz član $0 \leq i \leq N$. Za $i = 0$ imamo $(\operatorname{div}^* U)_i = -N U_0$, za $i = N + 1$ imamo $(\operatorname{div}^* U)_{N+1} = N U_N$, a za $i = 1, \dots, N$ vrijedi $(\operatorname{div}^* U)_i = N(U_{i-1} - U_i)$. Kada div^* komponiramo s M imamo $(M(\operatorname{div}^* U))_0 = (M(\operatorname{div}^* U))_{N+1} = 0$, te $(M(\operatorname{div}^* U))_i = N(U_{i-1} - U_i)$ za $i = 1, \dots, N$.

Za operator div u slučaju $i = 0$ imamo

$$\begin{aligned} (\operatorname{div}(M(\operatorname{div}^* U)))_0 &= N((M(\operatorname{div}^* U))_1 - (M(\operatorname{div}^* U))_0) \\ &= N(N(U_0 - U_1) - 0) = N^2(U_0 - U_1), \end{aligned}$$

u slučaju $i = N$ imamo

$$\begin{aligned} (\operatorname{div}(M(\operatorname{div}^* U)))_N &= N((M(\operatorname{div}^* U))_{N+1} - (M(\operatorname{div}^* U))_N) \\ &= N(0 - N(U_{N-1} - U_N)) = N^2(U_N - U_{N-1}), \end{aligned}$$

a u slučaju $i = 1, \dots, N - 1$ vrijedi

$$(\operatorname{div}(M(\operatorname{div}^* U)))_i = N(N(U_i - U_{i+1}) - N(U_{i-1} - U_i)) = N^2(2U_i - U_{i-1} - U_{i+1}).$$

Analogno vrijedi za y -os (skaliramo s faktorom P^2) i za t -os (skaliramo s faktorom Q^2).

Neka je sada $L^{(n)} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ linearan operator, koji za svaki vektor $v \in \mathbb{R}^n$ djeluje upravo kako i $(\operatorname{div}(M(\operatorname{div}^* \cdot)))$ po linijama, odnosno kada fiksiramo dva indeksa. Sada za $L^{(n)}$ vrijede iste formule koje smo gore izveli za $(\operatorname{div}(M(\operatorname{div}^* \cdot)))$, bez faktora N^2 . Također, primijetimo da je $L^{(n)}$ diskretni 1D Laplaceov operator s Neumannovim rubnim uvjetima.

Proširimo njegovu definiciju tako da djeluje na prostoru $\mathbb{R}^{D_C}$. Neka je sada $L : \mathbb{R}^{D_C} \rightarrow \mathbb{R}^{D_C}$ definiran, po komponentama, sa

$$L_x := L^{(N+1)} \otimes \operatorname{Id}_{P+1} \otimes \operatorname{Id}_{Q+1},$$

$$L_y := \operatorname{Id}_{N+1} \otimes L^{(P+1)} \otimes \operatorname{Id}_{Q+1},$$

$$L_t := \operatorname{Id}_{N+1} \otimes \operatorname{Id}_{P+1} \otimes L^{(Q+1)},$$

gdje je $\otimes$ tenzorski produkt, a Id_n jedinična matrica dimenzije $n \times n$.

Sada možemo napisati izraz za $\operatorname{div}|_{\operatorname{Im} M}(\operatorname{div}^*|_{\operatorname{Im} M}(\cdot))$. Vrijedi

$$\operatorname{div}|_{\operatorname{Im} M}(\operatorname{div}^*|_{\operatorname{Im} M}(\cdot)) = N^2 L_x + P^2 L_y + Q^2 L_t,$$

gdje svaki L djeluje, kako je gore opisano, netrivialno samo duž svoje osi, dok duž ostalih osi djeluje kao identiteta. Primijetimo da je to zapravo diskretni 3D Laplaceov operator s Neumannovim rubnim uvjetima.

5.3.2. Kod za osnovne operatore

Prije nastavka, prikažimo kod za div , div^* te M . Operator div implementiran je tako da svaka dretva računa vrijednost u jednoj točki $(i, j, k) \in D_C$.

```
__global__ void div_kernel(GridDims g, const double* mx_s,
                           const double* my_s, const double* f_s,
```

```

double* out_c) {

long long n = g.size_c();
long long tid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
if (tid >= n) return;

int i, j, k;
decode_c(g, tid, i, j, k);
double dx = g.N*(mx_s[idx_mx(g, i + 1, j, k)] -
                mx_s[idx_mx(g, i, j, k)]);
double dy = g.P*(my_s[idx_my(g, i, j + 1, k)] -
                my_s[idx_my(g, i, j, k)]);
double dt = g.Q*(f_s[idx_f(g, i, j, k + 1)] -
                f_s[idx_f(g, i, j, k)]);
out_c[tid] = dx + dy + dt;
}

```

Adjungirani operator div^* implementira se, kako je spomenuto i u samom izvodu, po komponentama. Prikažimo kod za komponentu $\mathbf{m}^x$. Ostale se komponente implementiraju analogno.

```

__global__ void div_adjoint_x_kernel(GridDims g, const double* phi_c,
double* out_mx_s) {

long long n = g.size_mx();
long long tid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
if (tid >= n) return;

int k = (int)(tid % g.nt_mx());
long long tmp = tid / g.nt_mx();
int j = (int)(tmp % g.ny_mx());
int a = (int)(tmp / g.ny_mx());
double val = 0.0;
if (a - 1 >= 0 && a - 1 <= g.N)
    val += g.N * phi_c[idx_c(g, a - 1, j, k)];
}

```

```

if (a >= 0 && a <= g.N)
    val -= g.N * phi_c[idx_c(g, a, j, k)];

out_mx_s[tid] = val;
}

```

Preostaje implementirati operator M . On rubove od $\mathbf{m}^x$, $\mathbf{m}^y$ i $\mathbf{f}^t$ prepisuje s vrijednošću nula, a rubove $\mathbf{f}$. Kod za komponentu $\mathbf{m}^x$:

```

__global__ void zero_boundary_mx_kernel(GridDims g, double* mx_s) {
    long long n = (long long)g.ny_mx() * g.nt_mx();
    long long tid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (tid >= n) return;

    int k = (int)(tid % g.nt_mx());
    int j = (int)(tid / g.nt_mx());
    mx_s[idx_mx(g, 0, j, k)] = 0.0;
    mx_s[idx_mx(g, g.N + 1, j, k)] = 0.0;
}

```

Zatim moramo rubove od $\mathbf{f}^t$ prepisati s početnom i konačnom gustoćom $\mathbf{f}_0$, $\mathbf{f}_1$. To je nužno u konstrukciji $\tilde{U}_{cl}$

```

__global__ void clamp_boundary_f_kernel(GridDims g, double* f_s,
                                         const double* f0,
                                         const double* f1) {
    long long n = (long long)g.nx_f() * g.ny_f();
    long long tid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (tid >= n) return;

    int j = (int)(tid % g.ny_f());
    int i = (int)(tid / g.ny_f());
    long long ij = (long long)i * g.ny_f() + j;
    f_s[idx_f(g, i, j, 0)] = f0[ij];
}

```

```
f_s[idx_f(g, i, j, g.Q + 1)] = f1[ij];
}
```

5.3.3. Spektralna dekompozicija operatora L .

Sada ćemo pronaći svojstvene vektore i pripadne svojstvene vrijednosti operatora L . Krećemo prvo od operatora $L^{(n)}$. Kao što smo već naveli, radi se o diskretnom 1D Laplaceovom operatoru s Neumannovim rubnim uvjetima. Njegova je spektralna dekompozicija dana u ovom radu [14]. Ovdje ćemo samo provesti potvrdu te dekompozicije.

Postavimo prvo $\theta_l := \frac{l\pi}{2n}$ za $l = 0, \dots, n-1$. Kandidat za svojstveni vektor $v^{(l)}$ će nam biti

$$v_i^{(l)} := \cos(\theta_l(2i+1)), \quad i = 0, \dots, n-1,$$

a pripadna svojstvena vrijednost $4 \sin^2(\theta_l)$. Dokažimo sada da su ovo valjani kandidati.

U slučaju $i = 1, \dots, n-2$, koristeći trigonometrijski identitet

$$\cos A + \cos B = 2 \cos\left(\frac{A+B}{2}\right) \cos\left(\frac{A-B}{2}\right)$$

i parnost kosinusa, imamo

$$v_{i-1}^{(l)} + v_{i+1}^{(l)} = 2 \cos(\theta_l(2i+1)) \cos(2\theta_l) = v_i^{(l)} \cos(2\theta_l).$$

Oдавде slijedi:

$$(Lv^{(l)})_i = 2v_i^{(l)} - v_{i-1}^{(l)} - v_{i+1}^{(l)} = 2v_i^{(l)}(1 - \cos(2\theta_l)) = 4 \sin^2(\theta_l)v_i^{(l)},$$

koristeći $1 - \cos(2\theta_l) = 2 \sin^2(\theta_l)$.

U slučaju $i = 0$, koristeći trigonometrijski identitet

$$\cos A - \cos B = 2 \sin\left(\frac{A+B}{2}\right) \sin\left(\frac{B-A}{2}\right)$$

imamo

$$\begin{aligned}(Lv^{(l)})_0 &= v_0^{(l)} - v_1^{(l)} = \cos(\theta_l) - \cos(3\theta_l) = 2 \sin(2\theta_l) \sin(\theta_l) \\ &= 2(2 \sin(\theta_l) \cos(\theta_l)) \sin(\theta_l) = 4 \sin^2(\theta_l) v_0^{(l)}.\end{aligned}$$

U slučaju $i = n - 1$, umjesto ponovnog računa, pokazat ćemo da vrijedi

$$v_{n-1-i}^{(l)} = (-1)^l v_i^{(l)}.$$

Vrijedi

$$\theta_l(2(n-1-i)+1) = \theta_l(2n-2i-1) = \theta_l 2n - \theta_l(2i+1) = l\pi - \theta_l(2i+1).$$

Koristeći

$$\cos(l\pi - x) = \cos(l\pi) \cos(x) + \sin(l\pi) \sin(x) = (-1)^l \cos(x)$$

dobivamo:

$$v_{n-1-i}^{(l)} = \cos(l\pi - \theta_l(2i+1)) = (-1)^l \cos(\theta_l(2i+1)) = (-1)^l v_i^{(l)}.$$

Sada imamo

$$\begin{aligned}(Lv^{(l)})_{n-1} &= v_{n-1}^{(l)} - v_{n-2}^{(l)} = (-1)^l v_0^{(l)} - (-1)^l v_1^{(l)} \\ &= (-1)^l (v_0^{(l)} - v_1^{(l)}) = (-1)^l 4 \sin^2(\theta_l) v_0^{(l)} \\ &= 4 \sin^2(\theta_l) v_{n-1}^{(l)}\end{aligned}$$

Ovime je dokaz gotov.

5.3.4. Rješavanje sustava $\operatorname{div}|_{\operatorname{Im} M}(\operatorname{div}^*|_{\operatorname{Im} M}(\lambda)) = \operatorname{div}(\tilde{U}_{cl})$

Prisjetimo se problema kojeg rješavamo. Želimo za zadani $\tilde{U}_{cl} \in \mathcal{E}_s$ izračunati

$$U = \tilde{U}_{cl} + D^* = \tilde{U}_{cl} - M(\operatorname{div}^*((\operatorname{div}|_{\operatorname{Im} M} \operatorname{div}^*|_{\operatorname{Im} M})^{-1} \operatorname{div}(\tilde{U}_{cl}))).$$

Ključni je korak riješiti sustav

$$\operatorname{div}|_{\operatorname{Im}M}(\operatorname{div}^*|_{\operatorname{Im}M}(\lambda)) = \operatorname{div}(\tilde{U}_{cl}),$$

gdje je $\lambda \in \mathbb{R}^{Dc}$. Provjerimo prvo je li zadovoljen uvjet rješivosti.

Odredimo prvo svojstvene vektore od $L_x = L^{(N+1)} \otimes \operatorname{Id}_{P+1} \otimes \operatorname{Id}_{Q+1}$. Neka su $\{v_x^l\}$, gdje je $l = 1, \dots, N+1$ svojstveni vektori $L^{(N+1)}$, $\{v_y^m\}$, za $m = 1, \dots, P+1$ od $L^{(P+1)}$ te $\{v_t^n\}$, za $t = 1, \dots, Q+1$ od $L^{(Q+1)}$. Vrijedi

$$L_x(v_x^l \otimes v_y^m \otimes v_t^n) = (L^{(N+1)}v_x^l) \otimes (\operatorname{Id}_{P+1}v_y^m) \otimes (\operatorname{Id}_{Q+1}v_t^n) = (\sigma_x(l)v_x^l) \otimes (v_y^m) \otimes (v_t^n),$$

za sve $m = 1, \dots, P+1$ i sve $n = 1, \dots, Q+1$ i gdje je $\sigma_x(l) \in \mathbb{R}$ svojstvena vrijednost za svojstveni vektor v_x^l .

Neka je sada $v^{(l,m,n)} = v_x^l \otimes v_y^m \otimes v_t^n$. Vrijedi

$$\begin{aligned} \operatorname{div}|_{\operatorname{Im}M}(\operatorname{div}^*|_{\operatorname{Im}M}(v^{(l,m,n)})) &= N^2 L_x v^{(l,m,n)} + P^2 L_y v^{(l,m,n)} + Q^2 L_t v^{(l,m,n)} \\ &= (N^2 \sigma_x(l) + P^2 \sigma_y(m) + Q^2 \sigma_t(n)) v^{(l,m,n)}. \end{aligned}$$

Vidimo da je $v^{(l,m,n)}$ svojstveni vektor od $\operatorname{div}|_{\operatorname{Im}M}(\operatorname{div}^*|_{\operatorname{Im}M}(\cdot))$ sa svojstvenom vrijednošću $\Lambda(l, m, n) := N^2 \sigma_x(l) + P^2 \sigma_y(m) + Q^2 \sigma_t(n)$. Prisjetimo se da je za $\theta_l := \frac{l\pi}{2(N+1)}$, gdje $l = 0, \dots, N$, svojstvena vrijednost $\sigma_x(l) = 4 \sin^2(\theta_l)$, a pripadni svojstveni vektor $v_i^{(l)} = \cos(\theta_l(2i+1))$ za $i = 0, \dots, N-1$. Sada za $l = m = n = 0$ svojstvena vrijednost $\Lambda(0, 0, 0) = 0$, a pripadni svojstveni vektor $v^{(0,0,0)} = \mathbf{1}$. To znači da ako za $\lambda \in \mathbb{R}^{Dc}$ uzmemo konstanto polje (svugdje ista vrijednost), djelovanjem operatora $\operatorname{div}|_{\operatorname{Im}M} \operatorname{div}^*|_{\operatorname{Im}M}$ dobit ćemo nulu. Dodatno vrijedi

$$\langle \operatorname{div}|_{\operatorname{Im}M} \operatorname{div}^*|_{\operatorname{Im}M} \lambda, \lambda \rangle = \langle M \operatorname{div}^*(\lambda), M \operatorname{div}^*(\lambda) \rangle = \|M \operatorname{div}^*(\lambda)\|^2 \geq 0,$$

odnosno $\operatorname{div}|_{\operatorname{Im}M} \operatorname{div}^*|_{\operatorname{Im}M}$ je pozitivno semidefinitan operator. S obzirom da su sve svojstvene vrijednosti ≥ 0 , zaključujemo da je jezgra točno jednodimenzionalna, razapeta konstantnim vektorom $\mathbf{1}$. Da bi sustav $\operatorname{div}|_{\operatorname{Im}M} \operatorname{div}^*|_{\operatorname{Im}M} \lambda = \operatorname{div}(\tilde{U}_{cl})$ bio rješiv, $\operatorname{div}(\tilde{U}_{cl})$ mora biti okomit na jezgru, odnosno mora vrijediti $\langle \operatorname{div}(\tilde{U}_{cl}), \mathbf{1} \rangle = 0$. Eksplicitnim raspisom te sume

dobivamo

$$\sum_{i,j,k} \operatorname{div}(\tilde{U}_{cl})_{i,j,k} = \sum_{j,k} N(\mathbf{m}_{N+1,j,k}^x - \mathbf{m}_{0,j,k}^x) + \sum_{i,k} P(\mathbf{m}_{i,P+1,k}^y - \mathbf{m}_{i,0,k}^y) + \sum_{i,j} Q(\mathbf{f}_{i,j,Q+1} - \mathbf{f}_{i,j,0}),$$

gdje su $\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}$ pripadna skalarna polja momenata i gustoće na pomaknutoj mreži. Budući da, po konstrukciji, $\tilde{U}_{cl}$ na rubovima ima upravo $b_0 = ((0,0), (0,0), \mathbf{f}_0, \mathbf{f}_1)$, prve dvije sume u gornjem izrazu iščezavaju, a posljednja suma jednaka je

$$Q \sum_{i,j} (\mathbf{f}_{i,j}^{(1)} - \mathbf{f}_{i,j}^{(0)}) = Q \left(\sum_{i,j} \mathbf{f}_{i,j}^{(1)} - \sum_{i,j} \mathbf{f}_{i,j}^{(0)} \right) = Q(1 - 1) = 0,$$

jer se radi o gustoćama. Time je potvrđen uvjet da ovaj sustav bude rješiv.

Sada ćemo opisati kako možemo riješiti naš sustav. Ponovno gledamo samo pojedine komponente mreže. Neka je $\{v^{(l)}\}$ ortogonalna baza svojstvenih vektora simetričnog operatora L_x (jer je $\operatorname{div}|_{\operatorname{Im} M} \operatorname{div}^*|_{\operatorname{Im} M}$ samo-adjungirani operator), s vlastitim vrijednostima $\sigma_x(l)$. Prvi je korak razviti $\operatorname{div}(\tilde{U}_{cl}) = \sum_l c(l)v^{(l)}$, gdje se koeficijenti dobivaju projekcijom

$$c(l) = \frac{\langle \operatorname{div}(\tilde{U}_{cl}), v^{(l)} \rangle}{\|v^{(l)}\|^2}.$$

Drugi je korak razviti $\lambda = \sum_l a(l)v^{(l)}$, uz $L_x \lambda = \sum_l a(l)\sigma_x(l)v^{(l)}$. Na kraju moramo samo izjednačiti odgovarajuće koeficijente $a(l)\sigma_x(l) = c(l)$. U prvom koraku, koristeći

$$\cos^2(x) = \frac{1 + \cos(2x)}{2}$$

imamo

$$\|v^{(l)}\|^2 = \sum_{i=0}^{N-1} \cos^2(\theta_l(2i+1)) = \frac{N}{2} + \frac{1}{2} \sum_{i=0}^{N-1} \cos(2\theta_l(2i+1)).$$

Za $l \neq 0$, drugi je zbroj realan dio geometrijskog reda kompleksnih eksponencijala s koracima $\frac{l\pi}{(N+1)}$ preko punog perioda od $N+1$ članova, a takav zbroj jednako-raspoređenih kutova preko cijelog perioda iščezava, odnosno imamo $\|v^{(l)}\|^2 = \frac{N+1}{2}$. Za $l = 0$ svi članovi su $\cos(0) = 1$ pa imamo $\|v^{(l)}\|^2 = N+1$. Analogno za ostale komponente mreže.

5.3.5. Diskretna kosinusova transformacija

Cilj nam je riješiti sustav $\text{div}|_{\text{Im}M}\text{div}^*|_{\text{Im}M}\lambda = \text{div}(\tilde{U}_{cl})$. Prvi je korak razviti $\text{div}(\tilde{U}_{cl})$ u bazi svojstvenih vektora $v_{i,j,k}^{(l,m,n)} = v_x^{(l)}v_y^{(m)}v_t^{(n)}$. Moramo izračunati koeficijente $B(l, m, n) = \langle \text{div}(\tilde{U}_{cl}), v^{(l,m,n)} \rangle$, što se zbog strukture problema svodi na tri neovisna 1D računa. Izračun $\langle \text{div}(\tilde{U}_{cl}), v^{(l)} \rangle = \sum_i c(l) \cos(\theta_l(2i + 1))$ koji se pojavljuje u prvom koraku opisanog algoritma je, do na normalizacijski faktor, diskretna kosinusova transformacija (DCT-II) ulaznog niza $\text{div}(\tilde{U}_{cl})$ u jednoj dimenziji (vidjeti [15]). U implementaciji ćemo se poslužiti bibliotekom cuFFT, koju je napisala tvrtka Nvidia, a služi za provođenje brze Fourierove transformacije na Nvidia grafičkim karticama. U kodu se ovaj korak implementira unaprijednim prolazom po svakoj osi.

```
run_forward_axis(plan_x_fwd_, outer_x, Mx_, inner_x, rhs_c, a); // x os
run_forward_axis_y(a, b);                                     // y os
run_forward_axis(plan_t_fwd_, outer_t, Mt_, inner_t, b, a);   // t os
```

Svaki `run_forward_axis` poziv izvodi tri pod-koraka. Prikažimo prvo kod same funkcije.

```
void PoissonFFTSolver::run_forward_axis(cufftHandle plan,
                                         long long outer, long long M,
                                         long long inner,
                                         const double* in,
                                         double* out) {

    cufftDoubleComplex* cplx =
        thrust::raw_pointer_cast(cplx_scratch_.data());
    double* real_pad = thrust::raw_pointer_cast(real_scratch_.data());
    dct2_mirror_pad_kernel<<<grid_for(outer * 2 * M * inner), kBlock>>>
        (in, real_pad, outer, M, inner);
    long long Mp1 = M + 1;

    if (inner == 1) {
        cufft_check(cufftExecD2Z(plan, real_pad, cplx), "execD2Z(inner=1)");
    } else {
```

```

    for (long long o = 0; o < outer; ++o) {
        double* r_o = real_pad + o * (2 * M) * inner;
        cufftDoubleComplex* c_o = cplx + o * Mp1 * inner;
        cufft_check(cufftExecD2Z(plan, r_o, c_o), "execD2Z(looped)");
    }
}

dct2_extract_kernel<<<grid_for(outer * M * inner), kBlock>>>
(cplx, out, outer, M, inner);
}

```

Prvi je korak izvesti funkciju `dct2_mirror_pad_kernel` koja zrcalno proširuje $x[0 \dots M - 1]$ na $y[0 \dots 2M - 1]$. Ovaj je korak potreban zbog implementacijskog detalja u samoj `cuFFT` biblioteci.

```

__global__ void dct2_mirror_pad_kernel(const double* x, double* y,
                                       long long outer, long long M,
                                       long long inner) {

    long long twoM = 2 * M;
    long long n = outer * twoM * inner;
    long long tid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (tid >= n) return;

    long long in_ = tid % inner;
    long long tmp = tid / inner;
    long long p = tmp % twoM;
    long long o = tmp / twoM;
    long long src = (p < M) ? p : (twoM - 1 - p);
    y[tid] = x[(o * M + src) * inner + in_];
}

```

Drugi je korak izvesti funkciju `cufftExecD2Z`. To je `cuFFT` funkcija koja provodi samu kosinusovu transformaciju. Konačno moramo izvesti `dct2_extract_kernel` koja izvlači izračunate (nenormalizirane) koeficijente. Njen kod:

```

__global__ void dct2_extract_kernel(const cufftDoubleComplex* Y,
                                   double* C, long long outer,
                                   long long M, long long inner) {
    long long n = outer * M * inner;
    long long tid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (tid >= n) return;

    long long in_ = tid % inner;
    long long tmp = tid / inner;
    long long k = tmp % M;
    long long o = tmp / M;
    long long Mp1 = M + 1;
    cufftDoubleComplex y = Y[(o * Mp1 + k) * inner + in_];
    double theta = -kPi * (double)k / (2.0 * (double)M);
    double c = cos(theta), s = sin(theta);
    C[tid] = 0.5 * (c * y.x - s * y.y);
}

```

Sada se u $C[k] = \frac{1}{2}\text{Re}(\exp(-i\pi k)/(2M))Y[k]$ nalazi (nenormalizirani) koeficijent za k . Na kraju izvođenja sve tri transformacije, spremnik a sadrži koeficijente $B(l, m, n)$. Sljedeći je korak provesti normalizaciju, odnosno svaki koeficijent podijeliti s izrazom $\|v^{(l,m,n)}\|^2 \Lambda(l, m, n)$, gdje je $\Lambda(l, m, n)$ svojstvena vrijednost pripadnog svojstvenog vektora $v^{(l,m,n)}$. Te vrijednosti računamo, po već izvedenim formulama, u sljedećem kodu.

```

auto lambda_norm = [](int divisions, long long M, long long l,
                      double& lam, double& norm) {
    double s = sin((double)l * kPi / (2.0 * (double)M));
    lam = (double)divisions * (double)divisions * 4.0 * s * s;
    norm = (l == 0) ? (double)M : (double)M / 2.0;
};

std::vector<double> lam_x(Mx_), norm_x(Mx_), lam_y(My_), norm_y(My_),
    lam_t(Mt_), norm_t(Mt_);

```

```

for (long long l = 0; l < Mx_; ++l)
    lambda_norm(g.N, Mx_, l, lam_x[l], norm_x[l]);
for (long long m = 0; m < My_; ++m)
    lambda_norm(g.P, My_, m, lam_y[m], norm_y[m]);
for (long long n = 0; n < Mt_; ++n)
    lambda_norm(g.Q, Mt_, n, lam_t[n], norm_t[n]);

```

Nakon ovog moramo primijeniti inverznu kosinusovu transformaciju kako bismo rekonstruirali $\lambda_{i,j,k} = \sum a(l,m,n) v_i^{(l)} v_j^{(m)} v_k^{(n)}$, gdje $a(l,m,n)$ sadrži sada normalizirani koeficijent. Ove inverzne transformacije, kao i unaprijedne, provodimo prolazom po svakoj osi.

```

run_inverse_axis(plan_t_inv_, outer_t, Mt_, inner_t, a, b);
run_inverse_axis_y(b, a);
run_inverse_axis(plan_x_inv_, outer_x, Mx_, inner_x, a, phi_c);

```

Kao i prije, svaki run_inverse-axis izvodi tri pod-koraka.

```

void PoissonFFTSolver::run_inverse_axis(cufftHandle plan,
                                         long long outer, long long M,
                                         long long inner,
                                         const double* in,
                                         double* out) {

    cufftDoubleComplex* cplx =
        thrust::raw_pointer_cast(cplx_scratch_.data());
    double* real_pad = thrust::raw_pointer_cast(real_scratch_.data());
    long long Mp1 = M + 1;
    dct3_embed_kernel<<<grid_for(outer * Mp1 * inner), kBlock>>>
        (in, cplx, outer, M, inner);
    if (inner == 1) {
        cufft_check(cufftExecZ2D(plan, cplx, real_pad),
                    "execZ2D(inner=1)");
    } else {
        for (long long o = 0; o < outer; ++o) {

```

```

    cufftDoubleComplex* c_o = cplx + o * Mp1 * inner;
    double* r_o = real_pad + o * (2 * M) * inner;
    cufft_check(cufftExecZ2D(plan, c_o, r_o), "execZ2D(looped)");
}
}
dct3_truncate_kernel<<<grid_for(outer * M * inner), kBlock>>>
(real_pad, out, outer, M, inner);
}

```

Prvi je korak izvesti funkciju `dct3_embed_kernel` koja ugrađuje izračunate koeficijente u spremniku a u oblik koji zahtjeva interna implementacija `cuFFT` funkcije `cufftExecZ2D` za inverznu kosinusovu transformaciju. Nakon toga provodi se sama inverzna transformacija prije spomenutom funkcijom, a nakon toga `dct3_truncate_kernel` koja zadržava prvih M rekonstruiranih uzoraka, s obzirom da je izlaz inverzne transformacije $2M$ uzoraka, gdje je druga polovica zrcalna slika prve pa se odbacuje. Kod za ove funkcije nećemo prikazivati radi preglednosti rada i jer su vrlo slične gore navedenim funkcijama. Konačni se rezultat, odnosno rješenje našeg sustava sada nalazi u spremniku `phi_c`.

5.3.6. Opis implementacije proksimalnog operatora

Prisjetimo se prvo kako izgleda konačna formula za ovaj proksimalni operator. Neka je njegovo rješenje $U \in \mathcal{E}_s$, a ulaz $\tilde{U} \in \mathcal{E}_s$. Konačna formula za U je sada

$$U = \tilde{U}_{cl} - M(\operatorname{div}^*((\operatorname{div}|_{\operatorname{Im}M} \operatorname{div}^*|_{\operatorname{Im}M})^{-1} \operatorname{div}(\tilde{U}_{cl}))),$$

gdje je $\tilde{U}_{cl} = M\tilde{U} + (\operatorname{Id} - M)B_0$ i $B_0 \in \mathcal{E}_s$ takav da na rubovima ima b_0 te je rješenje sustava $\operatorname{div}|_{\operatorname{Im}M} \operatorname{div}^*|_{\operatorname{Im}M} \lambda = \operatorname{div}(\tilde{U}_{cl})$. Prvi korak je izračunati $\tilde{U}_{cl}$, što možemo lako učiniti implementacijom operatora M i funkcijama `zero_boundary_mx_kernel`, `zero_boundary_my_kernel` te `clamp_boundary_f_kernel`. Idući korak je izračunati $\operatorname{div}(\tilde{U}_{cl})$, također lagano korištenjem implementiranog operatora `div`. Dalje, sustav $\operatorname{div}|_{\operatorname{Im}M} \operatorname{div}^*|_{\operatorname{Im}M} \lambda = \operatorname{div}(\tilde{U}_{cl})$ rješavamo diskretnom kosinusovom transformacijom opisanom u prijašnjem odjeljku. Nakon toga računamo $M(\operatorname{div}^*(\lambda))$ korištenjem implementiranih operatora `div*` te M . Posljednji je korak provesti paralelizirano oduzimanje $\tilde{U}_{cl} - M(\operatorname{div}^*(\lambda))$. Time je ovaj proksimalni operator u potpunosti implementiran.

5.4. Implementacija proksimalnog operatora od $\iota_{C_c,s}$

Proksimalni operator koji želimo implementirati svodi se na formu

$$\text{Proj}_{C_{c,s}}(\tilde{Z}) = (\text{Id} - B^*(BB^*)^{-1}B)\tilde{Z},$$

gdje su $Z, \tilde{Z} \in H$. Njegova je uloga osigurati da vrijedi uvjet $I(U) = V \in \mathcal{E}_c$. Za njegovu implementaciju moramo znati kako izgleda BB^* , za $B = [I \mid -\text{Id}_{\mathcal{E}_c}]$, gdje je $I : \mathcal{E}_s \rightarrow \mathcal{E}_c$ operator interpolacije sa skupa pomaknutih mreža na skup centriranih mreža, a $\text{Id}_{\mathcal{E}_c} : \mathcal{E}_c \rightarrow \mathcal{E}_c$ identiteta.

5.4.1. Implementacija operatora I

Operator interpolacije I ne miješa komponente $\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}$, pa izvod adjungiranog operatora I^* možemo provesti po komponentama. Točnije, operator interpolacije, definiran po komponentama $(\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t)$ pomaknute mreže i točkama $(i, j, k) \in D_C$, tj. $0 \leq i \leq N, 0 \leq j \leq P, 0 \leq k \leq Q$ glasi:

$$\begin{aligned} I(U)_{i,j,k}^x &= \frac{\mathbf{m}_{i,j,k}^x + \mathbf{m}_{i+1,j,k}^x}{2}, \\ I(U)_{i,j,k}^y &= \frac{\mathbf{m}_{i,j,k}^y + \mathbf{m}_{i,j+1,k}^y}{2}, \\ I(U)_{i,j,k}^t &= \frac{\mathbf{f}_{i,j,k}^t + \mathbf{f}_{i,j,k+1}^t}{2}, \end{aligned}$$

gdje je $U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_s$ zapis po komponentama pomaknute mreže, a $I(U)^x \mathbf{m}^x$ komponenta interpolirane centrirane mreže.

Neka je sada $\tilde{I}^{(n)} : \mathbb{R}^{n+1} \rightarrow \mathbb{R}^n$ linearni operator interpolacije koji za svaki vektor $v \in \mathbb{R}^{n+1}$ djeluje formulom

$$(\tilde{I}^{(n)}v)_i = \frac{v_i + v_{i+1}}{2},$$

za sve $i = 0, \dots, n-1$. Ako sa $I^x : \mathbb{R}^{D_s^x} \rightarrow \mathbb{R}^{D_c}$ označimo komponentu operatora I koja

djeluje samo na $\mathbf{m}^x$, te analogno I^y, I^t koji djeluju samo na $\mathbf{m}^y, \mathbf{f}^t$, vrijedi

$$I^x := \tilde{I}^{(N+1)} \otimes \text{Id}_{P+1} \otimes \text{Id}_{Q+1},$$

$$I^y := \text{Id}_{N+1} \otimes \tilde{I}^{(P+1)} \otimes \text{Id}_{Q+1},$$

$$I^t := \text{Id}_{N+1} \otimes \text{Id}_{P+1} \otimes \tilde{I}^{(Q+1)}.$$

Operator I možemo zapisati kao

$$I = I^x \oplus I^y \oplus I^t,$$

, gdje je $\oplus$ direktna suma operatora Po komponentama imamo, za $U = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}^t) \in \mathcal{E}_s$,

$$I(U) = (I^x \mathbf{m}^x, I^y \mathbf{m}^y, I^t \mathbf{f}^t).$$

Implementacija operatora interpolacije I jednostavno se može paralelizirati tako da se svakoj točki centrirane mreže dodijeli jedna dretva

```
__global__ void interp_kernel(GridDims g, const double* mx_s,
                             const double* my_s, const double* f_s,
                             double* mx_c, double* my_c, double* f_c) {
    long long n = g.size_c();
    long long tid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (tid >= n) return;

    int i, j, k;
    decode_c(g, tid, i, j, k);
    mx_c[tid] = 0.5*(mx_s[idx_mx(g, i, j, k)] +
                    mx_s[idx_mx(g, i + 1, j, k)]);
    my_c[tid] = 0.5*(my_s[idx_my(g, i, j, k)] +
                    my_s[idx_my(g, i, j + 1, k)]);
    f_c[tid] = 0.5*(f_s[idx_f(g, i, j, k)] +
                    f_s[idx_f(g, i, j, k + 1)]);
}
```

gdje koristimo pomoćnu funkciju `decode_c` koja dekodira indeks dretve u indekse (i, j, k) na centriranoj mreži.

```
__host__ __device__ inline void decode_c(const GridDims& g,
                                         long long tid,
                                         int& i, int& j, int& k) {

    k = (int)(tid % g.nt_c());
    long long tmp = tid / g.nt_c();
    j = (int)(tmp % g.ny_c());
    i = (int)(tmp / g.ny_c());
}
```

5.4.2. Implementacija operatora $\tilde{\mathcal{I}}^*$

Dalje tražimo $\mathcal{I}^* : \mathcal{E}_c \rightarrow \mathcal{E}_s$. Neka je $\tilde{\mathcal{I}}^{*(n)} : \mathbb{R}^n \rightarrow \mathbb{R}^{n+1}$ adjungirani operator od $\tilde{\mathcal{I}}^{(n)}$. Za proizvoljne vektore $v \in \mathbb{R}^{n+1}$ i $w \in \mathbb{R}^n$ mora vrijediti

$$\langle \tilde{\mathcal{I}}^{(n)} v, w \rangle = \langle v, \tilde{\mathcal{I}}^{*(n)} w \rangle.$$

Raspisivajući lijevu stranu dobivamo

$$\begin{aligned} \langle \tilde{\mathcal{I}}^{(n)} v, w \rangle &= \sum_{i=0}^{n-1} w_i \frac{v_i + v_{i+1}}{2} \\ &= \frac{1}{2} \sum_{i=0}^{n-1} w_i v_i + \frac{1}{2} \sum_{i=0}^{n-1} w_i v_{i+1} \\ &= \frac{1}{2} \sum_{i=0}^{n-1} w_i v_i + \frac{1}{2} \sum_{i=1}^n w_{i-1} v_i. \end{aligned}$$

Izjednačavajući koeficijente uz odgovarajuće indekse, dobivamo

$$(\tilde{\mathcal{I}}^{*(n)} w)_i = \begin{cases} \frac{w_0}{2}, & i = 0, \\ \frac{w_{i-1} + w_i}{2}, & i = 1, \dots, n-1, \\ \frac{w_{n-1}}{2}, & i = n. \end{cases}$$

Adjungirani operator interpolacije $I^* : \mathcal{E}_c \rightarrow \mathcal{E}_s$ sada možemo, po komponentama, zapisati kao

$$\begin{aligned} I^{*x} &:= \tilde{I}^{*(N+1)} \otimes \text{Id}_{P+1} \otimes \text{Id}_{Q+1}, \\ I^{*y} &:= \text{Id}_{N+1} \otimes \tilde{I}^{*(P+1)} \otimes \text{Id}_{Q+1}, \\ I^{*t} &:= \text{Id}_{N+1} \otimes \text{Id}_{P+1} \otimes \tilde{I}^{*(Q+1)}, \end{aligned}$$

pa je, za $V = (\mathbf{m}^x, \mathbf{m}^y, \mathbf{f}) \in \mathcal{E}_c$,

$$I^*(V) = (I^{*x}\mathbf{m}^x, I^{*y}\mathbf{m}^y, I^{*t}\mathbf{f})$$

Adjungirani operator interpolacije I^* analogno implementiramo kao i I , tako da se poziva po svakoj točki pomaknute mreže, uzimajući u obzir komponentu koju interpoliramo. Kod za komponentu $\mathbf{m}^x$ prikazan je u nastavku.

```
__global__ void interp_adjoint_x_kernel(GridDims g,
                                       const double* psi_mx_c,
                                       double* out_mx_s) {

    long long n = g.size_mx();
    long long tid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (tid >= n) return;

    int k = (int)(tid % g.nt_mx());
    long long tmp = tid / g.nt_mx();
    int j = (int)(tmp % g.ny_mx());
    int a = (int)(tmp / g.ny_mx());
    double val = 0.0;
    if (a - 1 >= 0 && a - 1 <= g.N)
        val += 0.5 * psi_mx_c[idx_c(g, a - 1, j, k)];
    if (a >= 0 && a <= g.N)
        val += 0.5 * psi_mx_c[idx_c(g, a, j, k)];
    out_mx_s[tid] = val;
}
```

Kod za komponente $\mathbf{m}^y$ i $\mathbf{f}$ analogan je gornjem, uz promjenu uloga pojedinih indeksa.

5.4.3. Izvod izraza za BB^*

Sada promatramo operator $B : H \rightarrow \mathcal{E}_c$ zadan s $B(U, V) = \mathcal{I}(U) - V$. Tražimo $B^* : \mathcal{E}_c \rightarrow H$ takav da za sve $(U, V) \in H$ i sve $W \in \mathcal{E}_c$ vrijedi

$$\langle B(U, V), W \rangle_C = \langle (U, V), B^*(W) \rangle_H$$

Raspisivanjem dobijemo

$$\begin{aligned} \langle B(U, V), W \rangle_C &= \langle \mathcal{I}(U), W \rangle_C - \langle V, W \rangle_C \\ &= \langle U, \mathcal{I}^*(W) \rangle_S + \langle V, -W \rangle_C \\ &= \langle (U, V), (\mathcal{I}^*(W), -W) \rangle_H \\ &= \langle (U, V), B^*(W) \rangle_H. \end{aligned}$$

Direktnim iščitavanjem dobijemo $B^*(W) = (\mathcal{I}^*(W), -W)$. Sada možemo eksplicitno odrediti BB^* . Primjenom kompozicije dobijemo

$$BB^*(W) = B(B^*(W)) = B(\mathcal{I}^*(W), -W) = \mathcal{I}(\mathcal{I}^*(W)) + W.$$

Preostaje nam odrediti $\mathcal{I}\mathcal{I}^*$. Korištenjem dobivenih izraza za $\tilde{\mathcal{I}}^{(n)}$ i $\tilde{\mathcal{I}}^{*(n)}$, za $w \in \mathbb{R}^n$

$$\tilde{\mathcal{I}}^{(n)}(\tilde{\mathcal{I}}^{*(n)}(w))_i = \frac{\tilde{\mathcal{I}}^*(w)_i + \tilde{\mathcal{I}}^*(w)_{i+1}}{2}.$$

U slučaju $i = 0$ imamo

$$\tilde{\mathcal{I}}^{(n)}(\tilde{\mathcal{I}}^{*(n)}(w))_0 = \frac{1}{2} \left(\frac{w_0}{2} + \frac{w_0 + w_1}{2} \right) = \frac{2w_0 + w_1}{4}.$$

U slučaju $1 \leq i \leq n - 2$ imamo

$$\tilde{\mathcal{I}}^{(n)}(\tilde{\mathcal{I}}^{*(n)}(w))_i = \frac{1}{2} \left(\frac{w_{i-1} + w_i}{2} + \frac{w_i + w_{i+1}}{2} \right) = \frac{w_{i-1} + 2w_i + w_{i+1}}{4}.$$

U slučaju $i = n - 1$ imamo

$$\tilde{I}^{(n)}(\tilde{I}^{*(n)}(w))_{n-1} = \frac{1}{2} \left(\frac{w_{n-2} + w_{n-1}}{2} + \frac{w_{n-1}}{2} \right) = \frac{w_{n-2} + 2w_{n-1}}{4}.$$

Primijetimo da operator $\tilde{I}^{(n)} \tilde{I}^{*(n)}$ možemo matrično zapisati kao

$$\tilde{I}^{(n)} \tilde{I}^{*(n)} = \begin{bmatrix} 1/2 & 1/4 & 0 & \cdots & 0 & 0 \\ 1/4 & 1/2 & 1/4 & \cdots & 0 & 0 \\ 0 & 1/4 & 1/2 & \ddots & 0 & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1/2 & 1/4 \\ 0 & 0 & 0 & \cdots & 1/4 & 1/2 \end{bmatrix}_{n \times n} =: T(n)$$

To je tridijagonalna matrica s $1/2$ na glavnoj dijagonali i $1/4$ na sporednim dijagonalama.

Iz dobivenih tenzorskih zapisa I i I^* , imamo

$$II^* = I^x I^{*x} \oplus I^y I^{*y} \oplus I^t I^{*t}.$$

Za $I^x I^{*x}$ imamo

$$\begin{aligned} I^x I^{*x} &= (\tilde{I}^{(N+1)} \otimes \text{Id}_{P+1} \otimes \text{Id}_{Q+1}) \cdot (\tilde{I}^{*(N+1)} \otimes \text{Id}_{P+1} \otimes \text{Id}_{Q+1}) \\ &= (\tilde{I}^{(N+1)} \tilde{I}^{*(N+1)}) \otimes \text{Id}_{P+1} \otimes \text{Id}_{Q+1} \\ &= T(N+1) \otimes \text{Id}_{P+1} \otimes \text{Id}_{Q+1}. \end{aligned}$$

Analogno za dvije ostale komponente.

Konačan izraz za II^* je sada

$$II^* = (T(N+1) \otimes \text{Id}_{P+1} \otimes \text{Id}_{Q+1}) \oplus (\text{Id}_{N+1} \otimes T(P+1) \otimes \text{Id}_{Q+1}) \oplus (\text{Id}_{N+1} \otimes \text{Id}_{P+1} \otimes T(Q+1)).$$

Za BB^* vrijedi

$$BB^* = II^* + \text{Id}_{\mathcal{E}_c}.$$

5.4.4. Thomasov algoritam

Podsjetimo se kako izgleda proksimalni operator koji želimo implementirati

$$\text{Proj}_{C_{c,s}}(\tilde{Z}) = (\text{Id} - B^*(BB^*)^{-1}B)\tilde{Z}.$$

Sustav koji želimo riješiti je

$$(BB^*)\lambda = B\tilde{Z},$$

gdje je $\tilde{Z} \in H$, a $\lambda \in \mathcal{E}_c$.

Primijetimo da je BB^* simetrična, tridijagonalna, strogo dijagonalno dominantna pa zato i pozitivno definitna matrica koja nezavisno djeluje po komponentama centrirane mreže. Zato ćemo u implementaciji koristiti Thomasov algoritam koji je za ovakvu matricu stabilan [16]. Thomasov se algoritam koristi za rješavanje tridijagonalnih sustava jednadžbi s n nepoznanica oblika

$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i.$$

Njegova je kompleksnost $O(n)$. Sastoji se unaprijednog prolaza, u kojem se računaju novi koeficijenti, i unazadnog prolaza u kojim se dobiva rješenje. Unaprijedni prolaz računa

$$c'_i = \begin{cases} \frac{c_i}{b_i}, & i = 1, \\ \frac{c_i}{b_i - a_i c'_{i-1}}, & i = 2, \dots, n-1 \end{cases}$$

i

$$d'_i = \begin{cases} \frac{d_i}{b_i}, & i = 1, \\ \frac{d_i - a_i d'_{i-1}}{b_i - a_i c'_{i-1}}, & i = 2, \dots, n-1. \end{cases}$$

Rješenje sustava dobivamo unazadnim prolazom

$$x_n = d'_n, \quad x_i = d'_i - c'_i x_{i+1}, \quad i = n-1, n-2, \dots, 1.$$

U našem slučaju, koeficijenti a_i, b_i, c_i unaprijed su poznati i konstantni. Za sve i za koje je odgovarajući koeficijent definiram imamo $a_i = \frac{1}{4}$, $b_i = \frac{3}{2}$, $c_i = \frac{1}{4}$. To nam omogućuje da na početku rada algoritma jednom unaprijed izračunamo koeficijente c'_i kojima onda po potrebi možemo pristupiti.

```
constexpr double kA = 0.25, kB = 1.5, kC = 0.25;
```

```
std::vector<double> precompute_cprime(int L) {
    std::vector<double> cprime(L, 0.0);
    cprime[0] = kC / kB;

    for (int i = 1; i < L - 1; ++i) {
        double denom = kB - kA * cprime[i - 1];
        cprime[i] = kC / denom;
    }

    return cprime;
}
```

Sada možemo implementirati sam Thomasov algoritam. Svaka će dretva riješiti sustav po liniji. Na primjer, za komponentu $\mathbf{m}^x$ fiksiramo (j, k) , a sama linija u ovom slučaju ima duljinu $N + 1$. Ukupno nam je potrebno $(P + 1)(Q + 1)$ dretvi da bismo riješili sustav za $\mathbf{m}^x$.

```
__global__ void thomas_solve_x_kernel(GridDims g, const double* cprime,
                                     const double* rhs, double* out) {
    long long lines = (long long)g.ny_c() * g.nt_c();
    long long lid = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (lid >= lines) return;
    int k = (int)(lid % g.nt_c());
    int j = (int)(lid / g.nt_c());
    int L = g.nx_c();

    double dprev = rhs[idx_c(g, 0, j, k)] / kB;
    out[idx_c(g, 0, j, k)] = dprev;
    for (int i = 1; i < L; ++i) {
        double denom = kB - kA * cprime[i - 1];
        double di = (rhs[idx_c(g, i, j, k)] - kA * dprev) / denom;
        out[idx_c(g, i, j, k)] = di;
    }
}
```

```

    dprev = di;
}

double xnext = out[idx_c(g, L - 1, j, k)];
for (int i = L - 2; i >= 0; --i) {
    double xi = out[idx_c(g, i, j, k)] - cprime[i] * xnext;
    out[idx_c(g, i, j, k)] = xi;
    xnext = xi;
}
}

```

Kod za komponente $\mathbf{m}^y$ i $\mathbf{f}$ analogan je gornjem, uz promjenu odgovarajućih indeksa.

5.4.5. Opis implementacije proksimalnog operatora

Sada se sam proksimalni operator za $\iota_{C_{c,s}}$ može lagano implementirati korištenjem gornjih funkcija. Neka je $\tilde{Z} = (\tilde{U}, \tilde{V}) \in H$ početni argument. Prvo računamo $B(\tilde{U}, \tilde{V}) = \mathcal{I}(\tilde{U}) - \tilde{V} \in \mathcal{E}_c$ korištenjem implementiranog operatora interpolacije i paraleliziranim oduzimanjem po svim točkama centrirane mreže. Zatim računamo $(BB^*)^{-1}(\mathcal{I}(\tilde{U}) - \tilde{V}) =: W \in \mathcal{E}_c$ koristeći Thomasov algoritam. Preostaje nam izračunati $(\tilde{U}, \tilde{V}) - B^*W = (\tilde{U}, \tilde{V}) - (\mathcal{I}^*(W), -W) \in H$. To implementiramo direktnim paraleliziranim zbrajanjem $\tilde{V} + W$ te, prvo primjenjujući adjungirani interpolacijski operator računamo $\mathcal{I}^*(W) \in \mathcal{E}_s$, a drugo paraleliziranim oduzimanjem $\tilde{U} - \mathcal{I}^*(W)$, gdje je $W = (BB^*)^{-1}(\mathcal{I}(\tilde{U}) - \tilde{V})$. Time je u potpunosti implementiran ovaj proksimalni operator.

5.5. Implementacija iteracije Douglas-Rachfordovog algoritma

U ovom odjeljku implementiramo jednu iteraciju Douglas-Rachfordovog algoritma. Čitatelj može matematički opis ove iteracije pronaći u odjeljku 4.4. Kod je prikazan u nastavku.

```

double dr_iteration_step(const JointLayout& L, const DRWeights& w,
                        ProxC& proxC, ProxConsensus& proxCS,
                        double* z1, double* z2, double* z3,

```



```

        double* x, double* p, double* pJ,
        double* pC, double* pCS,
        const double* f0, const double* f1) {
const GridDims& g = L.g;
const size_t bytes = (size_t)L.total * sizeof(double);
const size_t u_bytes = (size_t)L.off_mx_c * sizeof(double);

cudaMemcpy(pJ, z1, u_bytes, cudaMemcpyDeviceToDevice);
launch_cubic_prox(g, L.mx_c(z1), L.my_c(z1), L.f_c(z1), w.gamma,
        L.mx_c(pJ), L.my_c(pJ), L.f_c(pJ));

cudaMemcpy(pC, z2, bytes, cudaMemcpyDeviceToDevice);
proxC.apply(L.mx_s(pC), L.my_s(pC), L.f_s(pC), f0, f1);

proxCS.apply(L.mx_s(z3), L.my_s(z3), L.f_s(z3), L.mx_c(z3),
        L.my_c(z3), L.f_c(z3), L.mx_s(pCS), L.my_s(pCS),
        L.f_s(pCS), L.mx_c(pCS), L.my_c(pCS), L.f_c(pCS));

launch_lincomb3(p, w.omega1, pJ, w.omega2,
        pC, w.omega3, pCS, L.total);

launch_dr_zupdate(z1, p, x, pJ, w.alpha, L.total);
launch_dr_zupdate(z2, p, x, pC, w.alpha, L.total);
launch_dr_zupdate(z3, p, x, pCS, w.alpha, L.total);

double residual = residual_p_minus_x(p, x, w.alpha, L.total);
launch_dr_xupdate(x, p, w.alpha, L.total);
return residual;
}

```

Varijable $z1$, $z2$, $z3$, x , p , pJ , pC , pCS sve su tipa $H = \mathcal{E}_s \times \mathcal{E}_c$. Varijabla $DRWeights$ w sadrži težine ω_1 , ω_2 , ω_3 , γ iz definicije iteracije, a $f0$, $f1$ su početna i konačna gustoća. Varijabla $bytes$ sadrži ukupnu veličinu jednog para (U, V) , a varijabla

u_bytes veličinu jedne varijable $U \in \mathcal{E}_s$. Objasnimo sada kako se pokreće svaki proksimalni operator.

Proksimalni operator za $\mathcal{J}_d$ djeluje samo nad V dijelom varijable z1 pa se njen U dio prvo kopira u pomoćnu varijablu pJ. Nakon toga se pokreće račun ovog operatora (koji se svodi na rješavanje kubnih polinoma) funkcijom `launch_cubic_prox`. Rješenje se nakon izvođenja ove funkcije nalazi u pJ.

Proksimalni operator za ι_{C_d} mora pristupiti cijeloj varijabli z2. Zato se ona cijela kopira u pomoćni spremnik pC prije pokretanja. Taj operator, kako je definirano u njegovoj implementaciji 5.3., mora izračunati $\tilde{U}_{cl}$, a to radi "in-place" odnosno prepisuje ulazni $U \in \mathcal{E}_s$ na mjestu. Ovaj operator ne mijenja $V \in \mathcal{E}_c$, ali je njegova vrijednost potrebna u kasnijem koraku, zato se i taj bio mora kopirati. Sam se operator pokreće pozivom metode `apply` razreda `proxC` koja izvodi sve korake za provedbu računa ovog operatora.

Proksimalni operator za $\iota_{C_{c,s}}$ mora pristupiti cijelom paru $(U, V) \in H$, ali rezultat zapisuje u odvojeni spremnik, odnosno pCS. Zato ovdje nije potrebno kopiranje ikakvih podataka.

Nakon izračuna svih potrebnih operatora moramo izračunati

$$p = \omega_1 \cdot p^1 + \omega_2 \cdot p^2 + \omega_3 \cdot p^3.$$

To se radi pozivom funkcije `launch_lincomb3`. To je jednostavna funkcija koja paralelizirano provodi gornji račun u svakoj točki mreže.

```
__global__ void lincomb3_kernel(double* out, double w1, const double* a,
                                double w2, const double* b, double w3,
                                const double* c, long long n) {
    long long i = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (i >= n) return;

    out[i] = w1 * a[i] + w2 * b[i] + w3 * c[i];
}
```

Dalje moramo ažurirati vrijednosti varijabli z1, z2, z3. To se radi s tri poziva funkcije `launch_dr_zupdate`, od kojih svaka ažurira jednu varijablu.

```

__global__ void dr_zupdate_kernel(double* z, const double* p,
                                const double* x, const double* pi,
                                double alpha, long long n) {
    long long i = blockIdx.x * (long long)blockDim.x + threadIdx.x;
    if (i >= n) return;

    z[i] += alpha * (2.0 * p[i] - x[i] - pi[i]);
}

void launch_dr_zupdate(double* z, const double* p, const double* x,
                      const double* pi,
                      double alpha, long long n) {
    if (n <= 0) return;

    dr_zupdate_kernel<<<grid_for(n), kBlock>>>(z, p, x, pi, alpha, n);
}

```

Dalje računamo rezidual funkcijom `residual_p_minus_x`, koja koristi staru vrijednost varijable `x`, a tek onda se ona ažurira naredbom `launch_dr_xupdate`. Time je implementirana jedna iteracija Douglas-Rachfordovog algoritma. Dodajmo još samo napomenu vezanu za kopiranja koja su spomenuta u ovom odjeljku. Iako bilo kakvo kopiranje podataka najčešće jako usporava izvedbu algoritama, u ovom slučaju se radi o `cudaMemcpyDeviceToDevice` kopiranju, odnosno sve se događa na grafičkoj kartici koje su dizajnirane kako bi mogle podnijeti ogroman tok podataka (stotine GB/s). Zato ta kopiranja u ovom kontekstu imaju neznatan doprinos na vrijeme izvođenja samog algoritma.

5.6. Implementacija petlje Douglas-Rachfordovog algoritma

U ovom odjeljku prikazat ćemo implementacijske detalje same petlje Douglas-Rachfordovog algoritma. Prije toga objasniti ćemo jednu optimizaciju koja se koristi u gotovoj implementaciji, a to su CUDA grafovi (više o njima ovdje [17]). Svaki put kada se poziva neka funkcija za izvođenje na grafičkoj kartici (`kernel<<<...>>>(...)`, `cudaMemcpyAsync...`) to se radi

preko procesora računala, odnosno procesor treba pripremiti argumente, upisati naredbu u driver, driver treba provjeriti valjanost poziva, upisati je u red čekanja GPU-a... Svaki takav poziv traje određeno vrijeme, a pošto takvih poziva ima mnogo i izvode se svaku iteraciju, dolazi do značajnog usporenja rada algoritma. Rješenje tog problema upravo je korištenje CUDA grafova. Zbog toga što je niz operacija uvijek isti, a argumenti samih funkcija nikada se ne mijenjaju iz iteracije u iteraciju, onda se niz poziva funkcija može jednom snimiti kao graf čvorova i veza, jednom instancirati, i zatim ponavljano pokretati jednim, jeftinim pozivom. Time se smanjuje komunikacija između grafičke kartice i procesora, što ubrzava rad algoritma.

CUDA graf implementira se kroz nekoliko koraka. Prvi je korak snimanje svih poziva s procesora na GPU tijekom izvršavanja jedne iteracije algoritma.

```
cudaStreamBeginCapture(cudaStreamPerThread,
                        cudaStreamCaptureModeThreadLocal);
dr_iteration_step_capturable(...);
cudaStreamEndCapture(cudaStreamPerThread, &g_captured);
```

Rezultat je `cudaGraph_t g_captured`, opis za izradu izvršive verzije grafa. Idući je korak instancirati upravo izvršivu verziju naredbom

```
cudaGraphInstantiate(&graph_exec_, g_captured, 0);
```

Ova naredba radi pripremu na driver-razini. Alociraju se interne strukture i provjerava se valjanost opisanog niza operacija. To je jednokratani trošak nakon kojeg se može koristiti gotov CUDA graf. Idući je korak upravo koristiti graf za pokretanje naredbi na grafičkoj kartici. To se radi naredbom `cudaGraphLaunch(im.graph_exec_, cudaStreamPerThread);`.

Pokažimo sada ostatak implementacije. Prvo definiramo strukturu `Impl` koja sadrži sve korištene objekte potrebne za ispravno izvođenje algoritma i izradu CUDA grafa.

```
struct OptimalTransportSolver::Impl {
    GridDims g;
    JointLayout L;
    OTParams params;
    ProxC proxC;
```

```

ProxConsensus proxCS;

thrust::device_vector<double> z1, z2, z3, x, p, pJ, pC, pCS;
thrust::device_vector<double> f0d, f1d;

...
};

```

Dalje implementiramo funkciju solve razreda `OptimalTransportSolver` koja sadrži samu petlju algoritma. Ona prima vektore `f0`, `f1`, početnu i konačnu gustoću nad kojima provodimo transport. Prvi je korak napraviti linearnu interpolaciju tih gustoća na pomaknutoj i centriranoj mreži. Ovo nije nužan korak za rad samog algoritma, ali daje dobre inicijalne vrijednosti koje ubrzavaju rad algoritma.

```

std::vector<double> host_x(im.L.total, 0.0);
for (int i = 0; i <= g.N; ++i) {
    for (int j = 0; j <= g.P; ++j) {
        long long ij = (long long)i * g.ny_c() + j;
        double v0 = f0[ij], v1 = f1[ij];
        for (int k = 0; k <= g.Q; ++k) {
            double t = (double)k / g.Q;
            host_x[im.L.off_f_c + idx_c(g, i, j, k)] = (1.0 - t)*v0 + t*v1;
        }
        for (int c = 0; c <= g.Q + 1; ++c) {
            double t = (c == 0) ? 0.0 :
                (c == g.Q + 1 ? 1.0 : ((double)c - 0.5) / g.Q);
            host_x[im.L.off_f_s + idx_f(g, i, j, c)] = (1.0 - t)*v0 + t*v1;
        }
    }
}

```

Nakon toga dolazi sama petlja algoritma.

```

if (residual >= im.params.tol && im.params.max_iter > 1) {
    if (im.params.use_cuda_graph && im.ensure_graph_ready(w)) {

```

```

used_cuda_graph = true;

const int K = std::max(1, im.params.cuda_graph_batch_size);
double* rsq =
    thrust::raw_pointer_cast(im.residual_sq_dev.data());
int it = 1;
int last_print = 0;

while (it < im.params.max_iter) {
    int batch = std::min(K, im.params.max_iter - it);
    for (int r = 0; r < batch; ++r)
        cudaGraphLaunch(im.graph_exec_, cudaStreamPerThread);

    double sumsq = 0.0;
    cudaMemcpyAsync(&sumsq, rsq, sizeof(double),
                   cudaMemcpyDeviceToHost,
                   cudaStreamPerThread);
    cudaStreamSynchronize(cudaStreamPerThread);
    residual = im.params.alpha * std::sqrt(sumsq);
    it += batch;
    iterations_run = it;

    if (im.params.verbose_every > 0 &&
        it - last_print >= im.params.verbose_every) {
        std::printf("[ot] iter %d
                     (batch=%d, graphed) residual %.6e\n",
                     it, batch, residual);
        last_print = it;
    }
    if (residual < im.params.tol) break;
}
} else {
    for (int it = 1; it < im.params.max_iter; ++it) {

```

```

    residual = dr_iteration_step(im.L, w, im.proxC,
                                im.proxCS, z1, z2, z3,
                                xp, pp, pJ, pC, pCS, f0d, f1d);

    iterations_run = it + 1;
    if (im.params.verbose_every > 0
        && (it % im.params.verbose_every == 0))
        std::printf("[ot] iter %d residual %.6e\n", it, residual);
    if (residual < im.params.tol) break;
}
}
}
}

```

Petlja se prekida ako se dostigne zadani broj iteracija ili ako se rezidual spusti ispod razine koju je korisnik zadao. Nakon izvođenja samog algoritma, spremaju se konačne vrijednosti **m** i **f** na centriranoj mreži, računa se Wassersteinova-2 udaljenost i korisniku se prikazuje rezultat izvođenja algoritma. Time je u potpunosti opisan rad implementiranog algoritma.

6. Numeričke simulacije

U ovom dijelu rada prikazat ćemo numeričke simulacije dobivene korištenjem ove implementacije, usporediti sa postojećim algoritmima implementiranim za CPU i prikazati neke zanimljive primjere.¹

6.1. Specifikacija računala

Svi su rezultati dobiveni na računalu opremljenim s: 13th Gen Intel i7-13700KF (16 jezgri, 24 dretve, maksimalna radna frekvencija 5.4 GHz), 32 GB DDR5 RAM (radna frekvencija 6400 MHz), NVIDIA GeForce RTX 4080 (arhitektura NVIDIA Ada Lovelace, 9728 CUDA jezgri, 16 GB GDDR6X VRAM), 1.82TB SSD. Operacijski je sustav Windows 10 Pro.

6.2. Validacija algoritma

Kako bismo validirali valjanost implementiranog algoritma, moramo provjeriti je li konačna vrijednost izračunate Wasserstein-2 udaljenosti $\mathcal{W}_2$ jednaka onoj koju nalaže teorija. Problem je taj što za općenite 2D distribucije ne postoji zatvorena formula za $\mathcal{W}_2$. Srećom, postoji eksplicitna formula u slučaju transporta između normalnih razdioba. Navedimo tu formulu u sljedećem teoremu.

Teorem 2 *Neka je $\mu \sim \mathcal{N}_n(\mathbf{m}_1, \Sigma_1)$ i $\nu \sim \mathcal{N}_n(\mathbf{m}_2, \Sigma_2)$, gdje su $\mathbf{m}_1, \mathbf{m}_2 \in \mathbb{R}^n$ vektori očekivanja, a $\Sigma_1, \Sigma_2 \in \mathbb{R}^{n \times n}$ matrice kovarijanci. Tada vrijedi*

$$\mathcal{W}_2^2(\mu, \nu) = \|\mathbf{m}_1 - \mathbf{m}_2\|_2^2 + \text{Tr}(\Sigma_1 + \Sigma_2 - 2(\Sigma_1^{\frac{1}{2}}\Sigma_2\Sigma_1^{\frac{1}{2}})^{\frac{1}{2}}).$$

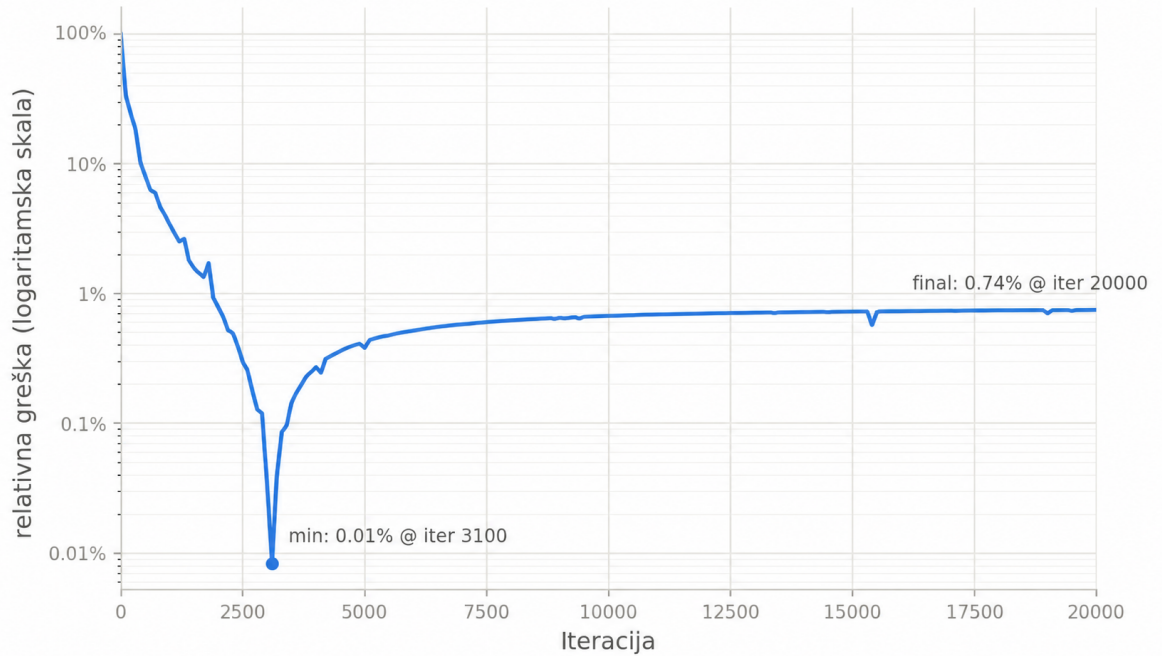
Dokaz zainteresirani mogu pronaći ovdje [3].

¹Link na GitHub repozitorij sa kodom: <https://github.com/Marko-Cindric/CUDA-DYNAMICAL-OT.git>

Primijetimo da se u slučaju jednakih matrica kovarijanci $\Sigma_1 = \Sigma_2$ formula pojednostavljuje na

$$\mathcal{W}_2^2(\mu, \nu) = \|\mathbf{m}_1 - \mathbf{m}_2\|_2^2.$$

Primijenimo sada formulu na konkretnom primjeru. Uzmimo dvije dvodimenzionalne normalne slučajne varijable s očekivanjima $\mathbf{m}_1 = (0.3, 0.5)$ i $\mathbf{m}_2 = (0.7, 0.5)$ te jednakim matricama kovarijanci. Eksplicitna nam formula daje $\mathcal{W}_2 = \|(0.3, 0.5) - (0.7, 0.5)\|_2 = 0.4$. Usporedimo tu vrijednosti s onom koju daje implementirani algoritam. Argumenti s kojim pokrećemo algoritam: gustoće dviju dvodimenzionalnih normalnih razdioba s gornjim očekivanjima i jednakim matricama kovarijanci $\Sigma = (0.0025, 0.0, 0.0, 0.0025)$, diskretizacijski parametri $N = P = Q = 64$ te brojem iteracija $2 \cdot 10^4$. Graf prikazuje ponašanje relativne gre-



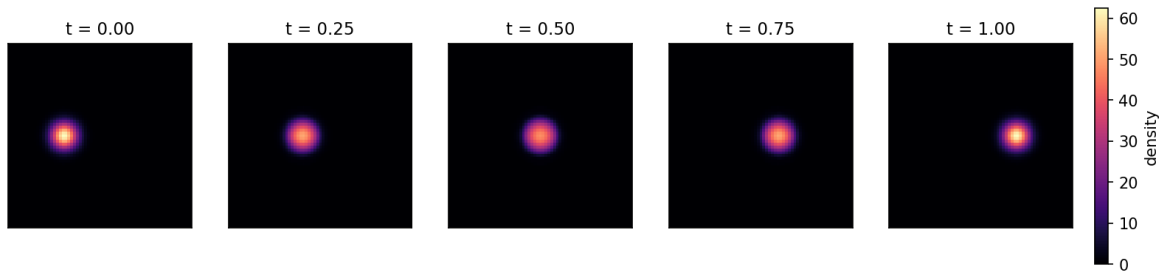
Slika 6.1. Relativna greška po iteracijama

ške (na logaritamskoj skali) u odnosu na trenutnu iteraciju algoritma. Nakon izvršenih $2 \cdot 10^4$ iteracija, izračunana udaljenost iznosi $\mathcal{W}_2 = 0.3970225402$, uz relativnu grešku od 0.74%. Pojasnimo sada mogući razlog za prisutnost greške. Gustoće dvodimenzionalnih normalnih slučajnih varijabli kontinuirane su te strogo pozitivne na cijeloj domeni $\mathbb{R}^2$, a ova implementacija razmatra samo njihove vrijednosti na diskretiziranom kvadratu $[0, 1]^2$. Posljedično, na ovom se primjeru ne provodi transport nad stvarnim gustoćama normalnih razdiobi, već nad njihovim diskretiziranim oblicima dodatno restringiranim na kvadrat $[0, 1]^2$. To je razlog zašto se izračunana udaljenost razlikuje od one teorijske i zašto se relativna greška nakon to-

liko iteracija stabilizira na 0.74%. Zato očekujemo da se relativna greška smanji korištenjem finije diskretizacije. Kao potvrdu ove teze, algoritam je pokrenut na istom primjeru, ali na dimenzijama $N = P = Q = 96$. Dobivena je udaljenost $\mathcal{W}_2 = 0.3980$ uz relativnu pogrešku od 0.51%. Prirodno je očekivati da će se daljnjim profinjenjem diskretizacije relativna greška dodatno smanjiti, stoga možemo s relativnom sigurnošću zaključiti da algoritam ispravno računa traženu udaljenost.

Komentirajmo ukratko i zanimljivu pojavu na gornjem grafu. Nakon 3100 iteracija relativna greška poprima svoj minimum od 0.01%. To ne znači da je to "čarobni" broj iteracija, već slučajnost da algoritam u tom trenutku ima najbližu procjenu stvarnoj vrijednosti udaljenosti.

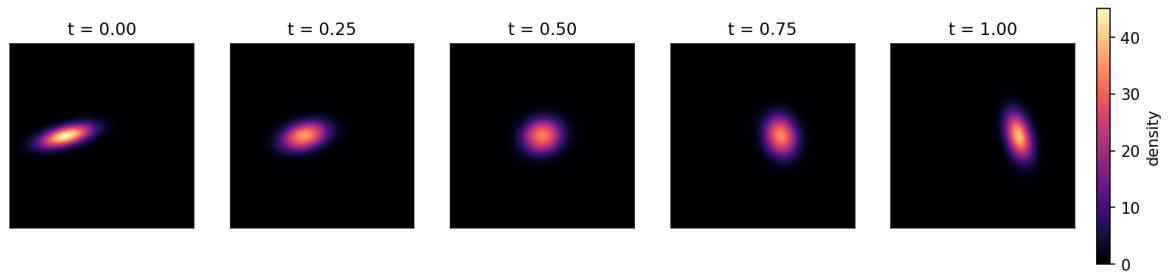
Prikažimo još vizualno (slika 6.2.) transport na ovom konkretnom primjeru. Dobijemo



Slika 6.2. Transport između dviju normalnih razdioba (ista kovarijanca)

točno ono što se očekuje kod transporta dviju normalnih razdioba s istom matricom kovarijance, gustoća zadržava svoj oblik i linearno se kreće s početne na konačnu poziciju.

Pokažimo još jedan primjer transporta dviju normalnih razdioba, ovaj put s različitim matricama kovarijanci. Prva ima očekivanje $\mathbf{m}_1 = (0.3, 0.5)$ i matricu kovarijance $\Sigma_1 = (0.009412, 0.002194, 0.002194, 0.001813)$, a druga očekivanje $\mathbf{m}_2 = (0.7, 0.5)$ i matricu kovarijance $\Sigma_2 = (0.002432, -0.001519, -0.001519, 0.007693)$. Parametri s kojim je pokrenut algoritam su $N = P = Q = 96$, a broj iteracija je $3 \cdot 10^4$. Teorijski očekivan rezultat, korištenjem eksplicitne formule, je $\mathcal{W}_2 = 0.40749$, a algoritmom dobivena vrijednost je $\mathcal{W}_2 = 0.40506$, relativna greška 0.60%. U nastavku je vizualizacija (slika 6.3.) samog transporta.



Slika 6.3. Transport između dviju normalnih razdioba (različita kovarijanca)

Tablica 6.1. Usporedba vremena izvođenja različitih implementacija (N je diskretizacijskih točaka, a broj do njega je broj iteracija)

Implementacija	$N = 32, 2 \cdot 10^4$	$N = 64, 3 \cdot 10^4$	$N = 96, 4 \cdot 10^4$
Matlab	137.829 s	4987.773 s	17383.508 s
C++	30.222 s	538.417 s	3365.546 s
CUDA C++ (naša implementacija)	3.468 s	33.389 s	186.240 s

6.3. Usporedba s postojećim implementacijama

Nakon uspješne provjere ispravnosti rada ove implementacije, usporedit ćemo ju s implementacijom istoga algoritma u programskom jeziku Matlab[11] i implementacijom u jeziku C++ (koja koristi višedretvenost na procesoru) [13]. Druge su dvije implementacije pisane isključivo za izvođenje na procesoru. Naglasak je na vremenskom ubrzanju postignutom uporabom grafičke kartice.

Ponovno koristimo gustoće dviju normalnih razdioba, prva s očekivanjem $\mathbf{m}_1 = (0.3, 0.5)$, a druga s $\mathbf{m}_2 = (0.7, 0.5)$, obje s istom matricom kovarijanca $\Sigma = (0.05, 0, 0, 0.05)$. Svaku implementaciju pokrećemo tri puta, s postepenim povećavanjem broja diskretizacijskih točaka (uz $N = P = Q$, gdje zbog čitkosti samo N piše u tablici) i broja iteracija. Sve tri implementacije rade u dvostrukoj preciznosti (double), s istim N, P, Q , istim brojem iteracija i istim hiper-parametrima α, γ i $\omega_1, \omega_2, \omega_3$ koje smo naveli na kraju četvrtog poglavlja. Konkretni parametri za N i broj iteracija, kao i vremena izvođenja prikazana su u tablici 6.1.

Iz tablice 6.1. vidimo da naša implementacija ostvaruje značajno ubrzanje izvođenja algoritma u odnosu na postojeće implementacije. Na najvećem primjeru ($N = P = Q = 96$ i $4 \cdot 10^4$ iteracija) vrijeme izvođenja smanjilo se 18.07 puta u odnosu na implementaciju u jeziku C++ i 93.34 puta u odnosu na implementaciju u Matlabu. Primijetimo da se potencijalno ubrzanje povećava dodavanjem broja iteracija i korištenjem diskretizacije veće razlu-

Tablica 6.2. Usporedba vremena izvođenja dijelova algoritma

Komponenta algoritma	Trajanje (ms)	% trajanja iteracije
$\text{Prox}_{\gamma \iota_{C_d}}$	0.874	$\approx 52\%$
ažuriranje varijabli i računanje reziduala	0.466	$\approx 22\%$
$\text{Prox}_{\gamma \iota_{C_{c,s}}}$	0.194	$\approx 12\%$
$\text{Prox}_{\gamma \mathcal{J}_d}$	0.145	$\approx 9\%$

čivosti. Napomenimo još da GeForce grafičke kartice, kakvu mi koristimo, imaju smanjenu propusnost u dvostrukoj preciznosti. Dobili bismo i veće ubrzanje da koristimo jednostruku preciznost (float), ali bi onda preciznost izračuna bila lošija.

Sada ćemo izmjeriti trajanje izvođenja pojedinih proksimalnih operatora kako bismo utvrdili koji je dio implementacije najsporiji. Rezultati su u tablici 6.2. Program je pokrenut s gustoćama normalnih razdioba koje su identične onima koje koristimo u validaciji od gore, te $N = P = Q = 64$ uz $2 \cdot 10^4$ iteracija. Iz tablice 6.2. vidimo da je najsporiji dio računanje proksimalnog operatora $\text{Prox}_{\gamma \iota_{C_d}}$. U prosjeku traje 0.874 milisekundi po iteraciji što je gotovo pola ukupnog vremena izvođenja jedne iteracije. To nije iznenađujuće jer je računanje diskretne kosinusove transformacije i njenog inverza jedan od najsloženijih dijelova cijelog algoritma. Tu su i pod-koraci koji su potrebni da se sama transformacija provede (izrada DCT planova). Iznenadjujuće je da je ažuriranje varijabli drugi najsporiji dio algoritma. Razlog tomu je što, iako su to sve obične linearne operacije koje se mogu maksimalno paralelizirati, one se u samoj izvedbi moraju provoditi sekvencijalno kako bismo dobili ispravan rezultat.

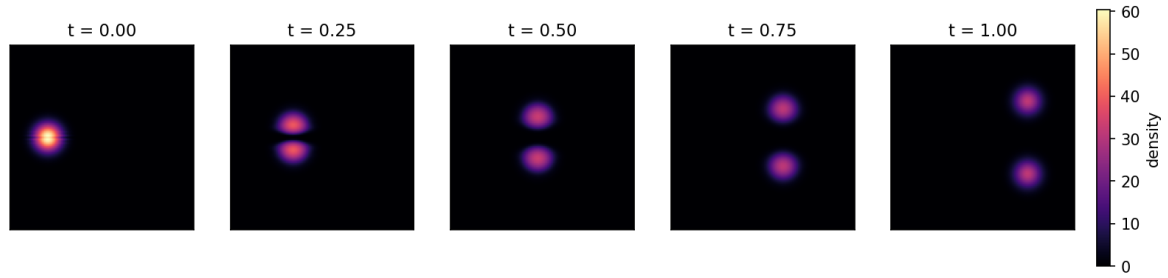
6.4. Različiti primjeri transporta

U ovom odjeljku prikazujemo dodatne primjere transporta dobivenih ovom implementacijom. S obzirom da za primjere koje ćemo prikazati u ovom dijelu rada ne postoji eksplicitna formula za Wassersteinovu-2 udaljenost, na njima ne možemo direktno provjeriti ispravnost rada algoritma, ali daju uvid na to kako se algoritam ponaša na kompleksnijim primjerima. Također ćemo prikazati primjere koji prikazuju konkretnu primjenu ovog algoritma.

6.4.1. Primjer cijepanja gustoće

Na ovom primjeru prikazujemo cijepanja gustoće. Za gustoću f_0 uzimamo gustoću obične dvodimenzionalne normalne slučajne varijable, a za gustoću f_1 uzimamo sumu gustoća dviju normalnih slučajnih varijabli. Na slici 6.4. vidimo da se početna gustoća razdvaja na dva

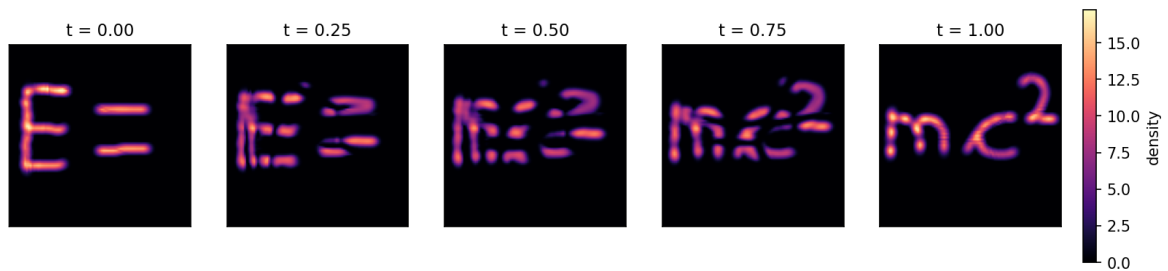
jednaka dijela od kojih se svaki transportira na korektno konačno odredište. Ovaj primjer pokazuje da naša implementacija dobro podnosi slučaj gdje se gustoće moraju razdvajati.



Slika 6.4. Cijepanje gustoće normalne razdiobe

6.4.2. Rukom crtane gustoće

U ovom primjeru smo rukom nacrtali gustoće. Prva je u obliku $E =$, a druga u obliku mc^2 . Na slici 6.5. možemo vidjeti transport gustoća. Ovaj primjer pokazuje da se algoritam dobro ponaša čak i kada su zadane gustoće proizvoljno određene.



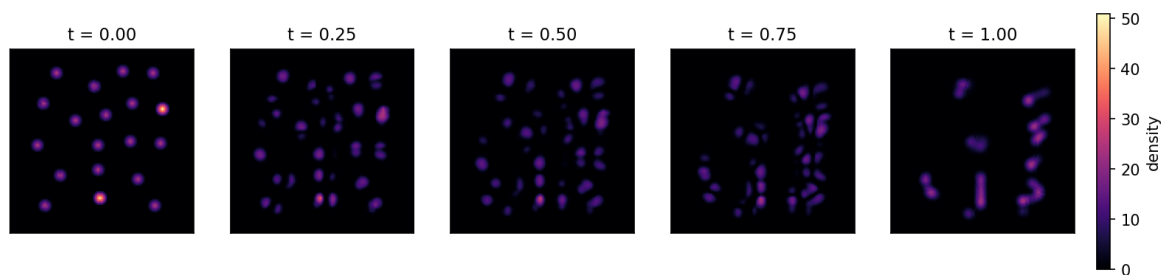
Slika 6.5. Transport $E =$ na mc^2

6.4.3. Neravnomjerno raspoređene gustoće

I na ovom primjeru se pokazuje transport rukom crtanih gustoća. Prva je mnogo malih nepravilno raspoređenih kružića, a druga mnogo nepravilnih, slučajno raspoređenih oblika. Transport je prikazan na slici 6.6. Na ovom primjeru možemo vidjeti da se algoritam dobro ponaša čak i kada su gustoće potpuno nepravilnog oblika i neravnomjerno raspoređene.

6.4.4. Transport između dviju crno-bijelih slika

Na ovom primjeru prikazujemo jednu zanimljivu primjenu ovog algoritma. Uzete su dvije crno-bijele slike (portreti Descartesa i Leibniza) i iz njih su dobivene gustoće tako da svjetliji



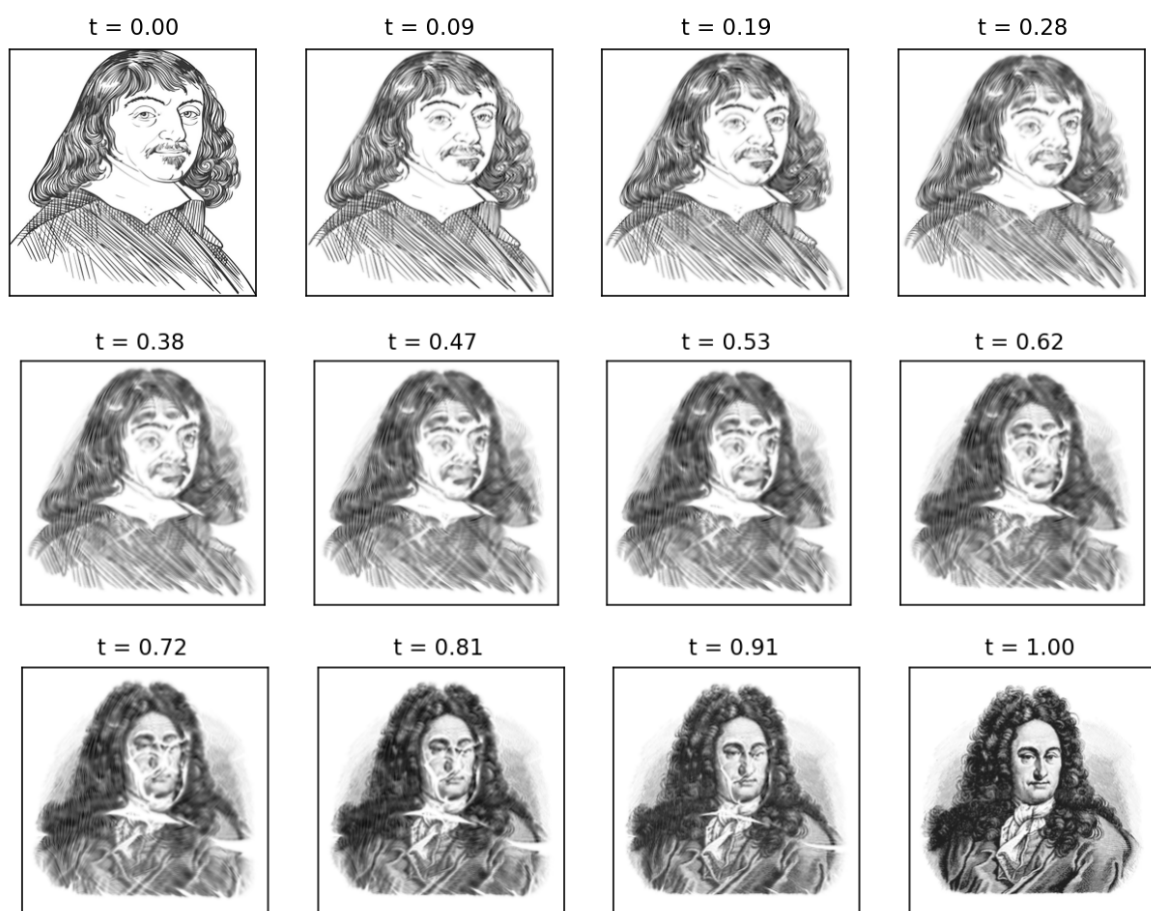
Slika 6.6. Transport nepravilnih neravnomjerno raspoređenih gustoća

pikseli imaju manju, a tamniji veću vrijednost gustoće. Transport između slika koristan je (i zanimljiv) iz više perspektiva.

Jedna od njih je ta, što se za razliku od linearne interpolacije slika koja jednostavno pretapa prvu u drugu (zbog čega jedna slika blijedi dok se druga pojavljuje), ovdje prva slika "fizički" pomiče na drugu. To daje puno uvjerljivije pretapanje.

Druga je primjena prijenos boja s jedne slike na drugu. Odaberemo izvorišnu sliku i njen histogram boja transportiramo na histogram boja ciljne slike. Rezultat je ciljna slika s "paletom" izvorišne [6].

Jedna od važnijih praktičnih primjena je korištenje optimalnog transporta na medicinskim slikama. Zadatak je naći deformaciju koja poravnava dvije snimke (CT prije i poslije zahvata, snimke istog pacijenta u različitim vremenskim trenucima...) Dinamička je formulacija tu iznimno korisna jer daje i cijelu putanju deformacije, što je korisno za modeliranje gibanja organa (npr. srca ili pluća) između dvije snimke. U radu [7] dan je primjer korištenja optimalnog transporta na primjeru neuronskog snimanja mozga.



Slika 6.7. Transport portreta Descartesa na portret Leibniza

7. Zaključak

U ovom je radu razvijena i implementirana paralelna metoda za numeričko rješavanje problema optimalnog transporta u dinamičkoj, Benamou-Brenierovoj formulaciji. Polazeći od Mongeove i Kantorovicheve formulacije, prikazana je ekvivalencija dinamičke formulacije s kvadratom L^2 -Wassersteinove udaljenosti, nakon čega je zamjenom varijabli $(v, f) \mapsto (m, f)$ dobiven konveksni optimizacijski problem s linearnim ograničenjima. Taj je problem diskretiziran na kombinaciji centrirane i pomaknute mreže, čime je omogućena konzistentna aproksimacija operatora divergencije središnjim konačnim razlikama, a zatim riješen asimetričnom inačicom Douglas-Rachfordovog algoritma uz konsenzus.

Središnji dio rada čini izvod i implementacija triju proksimalnih operatora. Proksimalni operator funkcionala $\mathcal{J}_d$ potpuno je separabilan i svodi se na traženje pozitivnog korijena kubne jednadžbe u svakoj točki mreže, što je riješeno Cardanovom formulom uz naknadno profinjenje Newtonovom metodom. Proksimalni operator indikatorske funkcije skupa C_d pokazao se ekvivalentnim rješavanju diskretne Laplaceove jednadžbe s Neumannovim rubnim uvjetima; iskorištena je spektralna dekompozicija pripadnog operatora, čime se rješavanje sustava svodi na trostruku diskretnu kosinusovu transformaciju, provedenu pomoću biblioteke cuFFT. Proksimalni operator indikatorske funkcije skupa $C_{c,s}$ vodi na simetričan, strogo dijagonalno dominantan tridijagonalan sustav koji se po linijama rješava Thomasovim algoritmom. Sve su komponente implementirane u jeziku CUDA C++, uz zajednički spremnik podataka (JointLayout) koji par $(U, V) \in H$ tretira kao jedan vektor te uz korištenje CUDA grafova, kojima je uklonjen značajan dio režijskog troška pokretanja jezgri na procesoru.

Ispravnost implementacije provjerena je na transportu između normalnih razdioba, za koje postoji eksplicitna formula za Wassersteinovu udaljenost. Relativna greška iznosila je 0.74% pri diskretizaciji $N = P = Q = 64$, odnosno 0.51% pri $N = P = Q = 96$, pri čemu preostala odstupanja potječu od ograničavanja inače neomeđenih gustoća na kvadrat

$[0, 1]^2$. Usporedbom s postojećim implementacijama istog algoritma u Matlabu i višedretvenom C++-u postignuto je ubrzanje od 93.34 odnosno 18.07 puta na najvećem razmatranom primjeru, uz naznaku da ubrzanje raste s finoćom diskretizacije i brojem iteracija. Profiliranjem je utvrđeno da približno polovicu vremena jedne iteracije troši proksimalni operator vezan uz jednadžbu kontinuiteta, što je i očekivano s obzirom na cijenu izravne i inverzne kosinusove transformacije.

Na nizu primjera pokazano je da se metoda robusno ponaša i izvan uvjeta u kojima postoji analitičko rješenje: kod cijepanja jedne gustoće na dvije, kod rukom crtanih i neravnomjerno raspoređenih gustoća te kod transporta između dviju crno-bijelih slika. Posljednji primjer ilustrira i praktičnu vrijednost dinamičke formulacije, koja uz konačno preslikavanje daje i cijelu putanju deformacije. Takvo je svojstvo osobito korisno u obradi slike i računalnoj grafici, primjerice kod prijenosa palete boja [6], te u medicinskoj primjeni, gdje se optimalni transport koristi za poravnavanje i usrednjavanje snimaka [7].

Literatura

- [1] D. W. Martinson. (2023., February) Optimal transport theory from sandcastles to artificial intelligence. Originally published in *Oxford Scientist Print* edition, Barriers, Michaelmas Term 2022. [Mrežno]. Adresa: <https://oxsci.org/optimal-transport-theory/>
- [2] F. Santambrogio, *Optimal Transport for Applied Mathematicians*, 1. izd., ser. Progress in Nonlinear Differential Equations and Their Applications. Birkhäuser Cham, 2015. <https://doi.org/10.1007/978-3-319-20828-2>
- [3] G. Peyré i M. Cuturi, “Computational optimal transport”, *Foundations and Trends in Machine Learning*, sv. 11, 2019. <https://doi.org/10.1561/22000000073>
- [4] Y. Brenier, “Polar factorization and monotone rearrangement of vector-valued functions”, *Communications on Pure and Applied Mathematics*, sv. 44, br. 4, str. 375–417, 1991. <https://doi.org/10.1002/cpa.3160440402>
- [5] J.-D. Benamou i Y. Brenier, “A computational fluid mechanics solution to the monge-kantorovich mass transfer problem”, *Numerische Mathematik*, sv. 84, br. 3, str. 375–393, 2000. <https://doi.org/10.1007/s002110050002>
- [6] N. Bonneel i J. Digne, “A survey of optimal transport for computer graphics and computer vision”, *Computer Graphics Forum*, sv. 42, br. 2, 2023. [Mrežno]. Adresa: <https://hal.archives-ouvertes.fr/hal-04029613>
- [7] A. Gramfort, G. Peyré, i M. Cuturi, “Fast optimal transport averaging of neuroimaging data”, u *Information Processing in Medical Imaging (IPMI)*, 2015., arXiv:1503.08596 [cs.CV]. [Mrežno]. Adresa: <https://doi.org/10.48550/arXiv.1503.08596>

- [8] T. Salimans, H. Zhang, A. Radford, i D. Metaxas, “Improving gans using optimal transport”, *arXiv preprint arXiv:1803.05573*, 2018. [Mrežno]. Adresa: <https://doi.org/10.48550/arXiv.1803.05573>
- [9] R. Bhardwaj, T. Vaidya, i S. Poria, “Towards solving nlp tasks with optimal transport loss”, *Journal of King Saud University - Computer and Information Sciences*, 2022. <https://doi.org/10.1016/j.jksuci.2022.10.031>
- [10] A. V. Kolesnikov, “Auctions and mass transportation”, *arXiv preprint arXiv:2312.08077*, 2024., version 2, updated June 4, 2024. [Mrežno]. Adresa: <https://arxiv.org/abs/2312.08077>
- [11] N. Papadakis, G. Peyre, i E. Oudet, “Optimal transport with proximal splitting”, *SIAM Journal on Imaging Sciences*, 2013. <https://doi.org/10.1137/130920058>
- [12] N. Parikh i S. Boyd, “Proximal algorithms”, *Foundations and Trends in Optimization*, sv. 1, br. 3, str. 127–239, 2014. <https://doi.org/10.1561/24000000003>
- [13] M. Cindrić, “Metoda proksimalnog cijepanja za kovarijancom modulirani optimalni transport”, 2025. [Mrežno]. Adresa: <https://urn.nsk.hr/urn:nbn:hr:168:780792>
- [14] G. Strang, “The discrete cosine transform”, *SIAM Review*, sv. 41, br. 1, str. 135–147, 1999. <https://doi.org/10.1137/S0036144598336745>
- [15] U. Schumann i R. A. Sweet, “Fast fourier transforms for direct solution of poisson’s equation with staggered boundary conditions”, *Journal of Computational Physics*, sv. 75, br. 1, str. 123–137, 1988.
- [16] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2. izd. Philadelphia: SIAM, 2002.
- [17] NVIDIA. (2026) Cuda programming guide: Cuda graphs. Section 4.2, last updated May 27, 2026. [Mrežno]. Adresa: <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html>

Sažetak

Numeričko rješavanje problema optimalnog transporta metodom proksimalnog cijepanja na grafičkoj kartici

Marko Cindrić

U radu se razmatra numeričko rješavanje problema optimalnog transporta u dinamičkoj, Benamou-Brenierovoj formulaciji, u kojoj se traženje L^2 -Wassersteinove udaljenosti između dviju gustoća svodi na konveksni optimizacijski problem s linearnim ograničenjem u obliku jednadžbe kontinuiteta. Problem se diskretizira na kombinaciji centrirane i pomaknute mreže te rješava asimetričnom inačicom Douglas-Rachfordovog algoritma, pri čemu se ciljni funkcional rastavlja na tri jednostavna funkcionala. Izvedeni su pripadni proksimalni operatori: prvi je potpuno separabilan i svodi se na rješavanje kubne jednadžbe u svakoj točki mreže, drugi na rješavanje diskretne Laplaceove jednadžbe s Neumannovim rubnim uvjetima pomoću diskretne kosinusove transformacije, a treći na rješavanje tridijagonalnog sustava Thomasovim algoritmom. Algoritam je implementiran u jeziku CUDA C++ i izvodi se na grafičkoj kartici, uz upotrebu biblioteke cuFFT i CUDA grafova. Ispravnost implementacije potvrđena je usporedbom s eksplicitnom formulom za Wassersteinovu udaljenost između normalnih razdioba, a mjerenja pokazuju ubrzanje do 18 puta u odnosu na višedretnu implementaciju u C++-u, odnosno do 93 puta u odnosu na implementaciju u Matlabu. Rad završava nizom primjera transporta, uključujući cijepanje gustoće, rukom crtane gustoće i transport između dviju crno-bijelih slika.

Ključne riječi: optimalni transport, Douglas-Rachford, proksimalni operator, Wasserstein, CUDA

Abstract

Numerical Solution of the Optimal Transport Problem Using the Proximal Splitting Method on a Graphics Card

Marko Cindrić

This thesis studies the numerical solution of the optimal transport problem in its dynamic, Benamou-Brenier formulation, in which computing the L^2 -Wasserstein distance between two densities reduces to a convex optimization problem constrained by the continuity equation. The problem is discretized on a combination of centered and staggered grids and solved by an asymmetric variant of the Douglas-Rachford algorithm, splitting the objective into three simple functionals. The corresponding proximal operators are derived: the first is fully separable and reduces to solving a cubic equation at every grid point, the second to solving a discrete Laplace equation with Neumann boundary conditions via the discrete cosine transform, and the third to solving a tridiagonal system by the Thomas algorithm. The algorithm is implemented in CUDA C++ and executed on a graphics card, using the cuFFT library and CUDA graphs. Correctness is verified against the closed-form expression for the Wasserstein distance between normal distributions, and the measurements show a speedup of up to 18 times over a multithreaded C++ implementation and up to 93 times over a Matlab implementation. The thesis concludes with several transport examples, including density splitting, hand-drawn densities, and transport between two black-and-white images.

Keywords: optimal transport, Douglas-Rachford, proximal operator, Wasserstein, CUDA